Aeroacoustic Performance of Toroidal Turbine Blades in Residential Areas

Ever Better Blades: Jose Gutierrez[1], Eileen Phan[1], Aidan Bryce Sabin[1], Sophia Shakir[1], & Aaron Tran[1]

[1] Department of Mechanical Engineering, University of Rochester, Rochester, NY

1. Abstract

Wind power is increasingly being used as a sustainable source of energy, and this increase in popularity has led to the implementation of wind turbines in residential areas. A common issue in residential wind turbines is the aerodynamic noise generated by their blades. Toroidal blades, which are commonly used in marine applications, could address this issue as they are quieter compared to conventional blade designs. However, there is little research on their application to wind turbines. The project sought to compare the power efficiency, sound pressure level (SPL), and tip-speed ratio between 3D printed conventional curved blades and toroidal blades. Testing was done indoors using a Dewalt leaf blower pointed at the blades attached to a shaft at the wind speeds of 4.25, 5.99, 7.90, and 9.68 m/s as measured by an anemometer. The voltage output was measured using an Arduino Uno REV3, noise data was collected with a microphone, and RPM was measured with a Hall sensor. It was found that the toroidal blades had higher efficiency and tip-speed ratio at the same input wind speed, while showing competitive acoustic behavior when compared at similar rotor speeds and when normalized by power output per unit airflow. Future experiments could attempt to compare the toroidal blades to a curved blade design with a higher surface area (such as by adding more blades), or to use a calibrated microphone for direct decibel comparison to regulatory noise limits in a specific residential area.

2. Introduction

The world's clean energy needs are higher than ever due to Earth's ever-increasing population. Research from S&P Global Sustainable [1] predicts that the demand for clean technology will grow by 30% in the next five years. Amongst this demand is the surge of artificial intelligence uptake, cloud computing, data centers, and the overall expansion of electrification and digitalization in rural and remote areas within the United States. With the steady depletion and negative environmental impacts of fossil fuels, the use of renewable energy will inevitably rise, giving way to the great promise that wind power holds for the future energy market [2].

Much of the research relating to turbine optimization focuses on energy efficiency; however, noise pollution is also an important design consideration that is often overlooked, especially as wind farms move closer to urban areas to supply power. The question then becomes how to retain power output while minimizing the noise that affects nearby residents. A survey conducted by Pawlaczyk-Łuszczyńska [3] of 517 people who lived in the vicinity of a wind farm found that 64.8% of respondents identified aerodynamic noise from rotor blades moving through the air, rather than mechanical noise from wind turbine generators, as the dominant source of wind-turbine noise. This has motivated research into a blade shape that minimizes aerodynamic noise generated from the flow around a turbine. Airfoil design is also important in optimizing power efficiency in wind turbines: symmetrical airfoils like the NACA 0018 are often used in wind turbine design as they are expected to produce consistent aerodynamic forces during blade rotation [4].

Considering common blade types for turbines, curved blades are known to have a greater energy output compared to straight ones due to their improved lift-to-drag ratio. However, there is

no study that investigates the noise level reduction that a curved blade offers. Furthermore, toroidal blades (see Figure 1) have been recently gaining popularity in underwater applications due to their increased propulsive efficiency, greater stability, and reduced noise output [5]. While their uses have been greatly explored in marine environments, there has yet to be extensive research on implementing toroidal blades in wind turbines. Hence, this is an unexplored idea in the pursuit of improving wind energy.

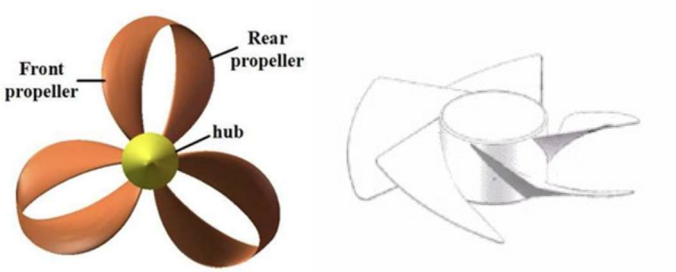


Figure 1: On the left is a geometric model of the toroidal propeller from the article by Bai [5]. On the right is a geometric model of a set of curved turbine blades investigated in the article by Zhu [2].

The following research aimed to compare the power efficiency (see Equation 2) and aerodynamic noise generation between two model turbine blade designs (curved and toroidal) using the NACA 0018 airfoil with the goal of assessing the best wind turbine design for urban applications. It was predicted that the toroidal blades would produce 20% less aerodynamic noise due to their reduced noise output in underwater applications, but that the toroidal blades would have a 15% lower power efficiency due to how curved blades are known to improve the lift-to-drag ratio.

3. Methods

3.1. Manufacturing Methods

To compare the power efficiency of the two blade designs, indoor trials were performed using a Dewalt leaf blower and cylindrical cardboard nozzle for uniform airflow. Air speed and the voltage generated by the blade rotation attached to a generator were used to find power efficiency and tip-speed ratio (TSR) versus wind speed for each blade design. To compare the aerodynamic noise generated, an input voltage was used to simulate the power efficiency trial speeds without the leaf blower and the isolated noise from the blades were compared for the two blade designs.

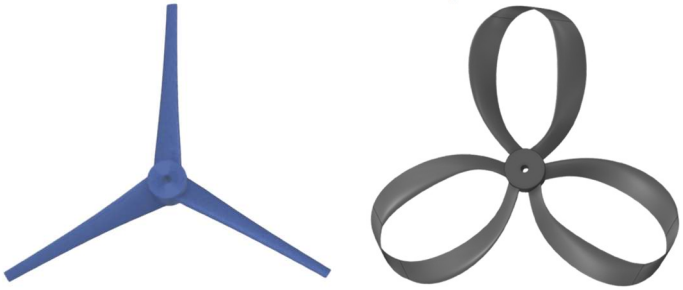


Figure 2: CAD models of the curved (left) and toroidal (right) blades.

The toroidal blades and their rotor were designed in Siemens NX and Onshape, whereas the curved blade design was created in QBlade to easily match the toroidal dimensions. Schmitz Optimization [6], a procedure computationally performed in QBlade, was used to optimize the curved blades' chord length to twist angle to account for wake rotation and wake losses. CAD

models of both blade designs are shown in Figure 2, and their drawings are shown in Appendix B, Figures 11 and 12.

Both designs use a NACA 0018 airfoil for their cross section and were exported to an STL file with a chordal tolerance of 0.1 mm and angular tolerance of 1°. Afterward, they were printed out of Bambu polylactic acid (PLA) Basic using a Bambu Lab H2D 3D printer at the Rettner Digital Media Lab. This printer's printing volume is 350x320x325 mm^3, and both designs were made to fit within these boundaries. After they were printed, both parts were dry sanded using 220 grit sandpaper, then wet sanded with 320 and then 400 grit (all available at Hopeman 124). Additionally, their holes were enlarged with an O drill bit so that the blades could fit onto the shaft of the motor. Table 1 lists information about certain parameters in both 3D models.

Table 1. Various information about the 3D printed blades.

Blade Type	Average Chord Length [mm]	Rotor Diameter [mm]	Moment of Inertia [$kgmm^2$]	Infill Type	Infill Density	Weight [oz]	Printing Time [hr]
Toroidal	28.97	356.50	1.08×10^3	Gyroid	15%	3.75	11.92
Curved	27.80	356.50	9.06×10^1	Rectilinear	100%	2.25	5.01

The average chord length was calculated from the lengths in Appendix B, Table 2; the similar average chord lengths helped maintain comparable blade-section dynamic similarity between the two model designs. The chord length is slightly different across the designs because, while the toroidal blades have a constant chord length, the chord length of the curved blades decreases as the blades extend out in accordance with QBlade's Schmitz Optimization. The models' rotor diameter, the diameter of the area swept by the blades, was reported by NX. The moment of inertia (MOI) was obtained using a Python script that calculated the signed volume of each model and generated an inertia tensor about the center of mass; the script used to perform this can be found in Appendix F. When working through design iterations, it was found that the toroidal blades' MOI was multiple orders of magnitude higher than the initial design for the curved ones. In an attempt to match their MOIs, the curved blades' infill was set to 100% to maximize its mass. The software for slicing the prints did not allow for a 100% gyroid infill, which is why rectilinear was used. In the end, the MOIs were still largely different (see Table 1) due to how the toroidal blades have a much higher mass. The weights were measured with a KitchenTour digital scale after the blades were fully processed (sanded and drilled).

A housing unit, Hall effect sensor mount, and magnet holders were designed in Siemens NX (see Appendix B) for each motor as well. The housing was made to minimize vibrations during testing, the Hall sensor mount kept the Hall sensor in place, and the magnet holders held the magnets onto the shaft. The Hall sensors were used to measure shaft RPM (further detailed in Section 3.3). All models were exported to an STL file with a chordal tolerance of 0.1 mm and angular tolerance of 1° then printed out of PLA using a Prusa MK4S 3D printer in Hopeman. These components can be seen in Figure 3.

An Adafruit QS-FS anemometer was purchased to measure wind speed. It has three cups which rotate to output an analog voltage that varies linearly with wind speed (from 0.4 to 2 V, which corresponds to 0 to 32.4 m/s) and an accuracy of ± (0.3 + 0.03v) m/s, where v is the present wind speed. A picture of it is in Appendix F. Additionally, two Melexis US5881 Hall sensors were purchased from Adafruit. They operate over a supply voltage range of 3.5 to 24 V.

Three 2"x4"x3' wood planks and four angle brackets were bought from Home Depot. Phillips #10 wood screws and two rubber P clamps with an inner diameter 2-1/16'' were obtained from McMaster-Carr. The P clamps were originally intended to act as motor housings, but they were

scrapped due to excessive vibrations during testing. Two Arduino Uno REV3's, two HUSETOO DC motors (shaft diameter of 8 mm), an uxcell lead shaft collar (diameter of 8 mm), eight neodymium magnets (3x2 mm), ELEGOO small breadboards, 20 cm breadboard jumper wires, and two FIFINE USB microphones were all acquired from Amazon. Finally, resistors and voltmeters were provided by the UR Mechanical Engineering Department, as well as the leaf blower used for indoor trials.

3.2. Experimental Setup

Each turbine base consists of a 2"x4"x2.5' wooden beam mounted onto a flat wooden 2'x2'x¼" foundation (cut with a miter saw in the Rettner Fabrication Lab) with angle brackets attached with #10 wood screws. The motor was secured on the top of the base with the motor housing to mitigate vibrational effects. The turbine's rotor diameter was approximately 0.3565 m and the distance from the ground to the turbine's hub was two times the rotor diameter (0.713 m) to reduce turbulent flow due to near wall effects from the ground. The base structure is visualized in Siemens NX as an assembly drawing (Appendix B, Figure 13).

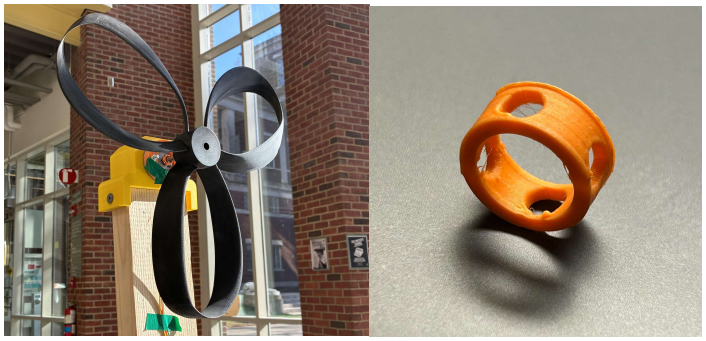


Figure 3: (Left): The toroidal blades attached to the shaft on top of its wooden tower. The smaller orange ring is the magnet holder, the larger orange ring is the Hall effect sensor mount, and the yellow unit is the motor housing. (Right): Close-up of the magnet holder (2D CAD drawing provided in Appendix B, Figure 15). It holds 4 magnets.

3.3. Testing Plan and Data Collection

Testing was conducted inside Rettner utilizing the leaf blower and a cardboard nozzle for uniform airflow. The cardboard nozzle used had a 98.2 cm length and a 40.15 cm average diameter. Both turbines were tested individually with the turbine base at the same distance from the leaf blower, and the wind speed at this distance was verified using the Adafruit anemometer.

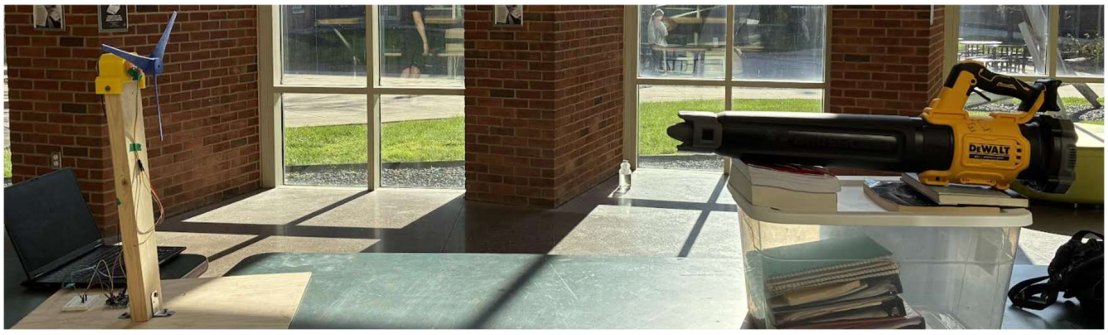


Figure 4: The experimental setup for the power efficiency trials featuring the curved turbine blades without the cardboard nozzle.

To find the wind speed, the voltage output of the anemometer was connected to an Arduino Uno REV3 (which has analog to digital conversion capabilities), and the wind speed was calculated

using a known linear scale from the QS-FS anemometer's website [7] relating the output voltage (0.4-2 V) to the wind speed range (0-32.4 m/s). Each of the turbine hubs had a bore in the back 24.4 mm deep. The DC generator shaft was then inserted into this bore and held in place with friction, and the rotation of the rotor caused the DC generator to output a voltage. The output of the DC generator was connected in series to a 1 kΩ resistive load, and the voltage across this load was measured using an Arduino Uno REV3. The circuit diagram is shown in Appendix F, Figure 17. The DC generator was mounted on top of the turbine tower. Four small neodymium magnets were placed opposite each other on the DC generator shaft, and a Hall sensor was mounted next to the DC generator (also on top of the tower) facing upward to trigger a voltage drop (logical LOW) each time a magnet passed by. The output of the Hall sensor went into one of the Arduino Uno REV3's external interrupt pins to track the RPM of the turbine (see Appendix F). The data tracked by the Arduino Uno REV3 was sent to its Serial Monitor and copied into a MATLAB script. These voltage output and RPM trials with the leaf blower each lasted one minute, and four trials were conducted for each wind speed.

Due to broadband noise from the leaf blower, noise trials were conducted separately from voltage output and RPM trials. Once more, trials were conducted for each turbine individually. The average RPM measured for each wind speed was recorded during power trials and then matched for noise trials by running the motor forwards using a power supply. A microphone was placed 10 mm below the tips of the rotating blades and connected through USB-C to a laptop to collect noise. The audio samples were recorded and stored in MATLAB (see Figure 5). Background noise was recorded in an extended four-minute trial without the turbine attached and with the motor running at the same RPM from the leaf blower trials. Four noise trials (each lasting one minute) were then taken at these RPMs generated with the power supply with the turbines attached.



Figure 5: The experimental setup used for the noise trials featuring the toroidal turbine blades.

3.4. Calculations and Data Analysis

Each trial is averaged using root mean square (RMS), such that the overall sound pressure level (SPL) can be found. Sound pressure level is given by

$$SPL = 20\log_{10}\left(\frac{P_{RMS}}{P_{ref}}\right) \quad (1)$$

where P_{rms} is the RMS sound pressure and P_{ref} is the reference pressure (20x10^{-6} Pa).

The power efficiency (η) for each turbine was calculated using the wind speed and voltage readings,

$$\eta = \frac{P_{resistor}}{P_{wind}} = \frac{\left(\frac{V^2}{R}\right)}{\left(\frac{1}{2}\rho A v^3\right)} \quad (2)$$

where V is the voltage across the load resistor; R is the resistance of the load resistor; ρ is the density of the air; A is the area swept by the turbine blades; υ is the velocity of the air. The tip-speed ratio (λ) was calculated using wind speed and Hall sensor readings,

$$\lambda = \frac{2\pi f r}{v} \quad (3)$$

where f is the frequency of the rotor shaft; r is the rotor radius; v is the velocity of the air. The tip-speed ratio is a dimensionless parameter that governs aerodynamic performance and indicates the operating point at which peak efficiency occurs, since Schmitz optimization depends primarily on the tip-speed ratio. MATLAB was used to generate plots of each turbine's efficiencies and tip-speed ratios during the duration of the experiment (see Appendix E).

Since efficiency and sound pressure level compare whole-rotor performance, and tip-speed ratio describes the whole rotor's operating condition, rotor diameter was used as the characteristic length for Reynolds number rather than airfoil chord length. This provides a global Reynolds number for turbine-scale flow, while chord-based Reynolds number remains more appropriate for local airfoil behavior. Therefore, the Reynolds Number (Re) for the reference turbine is given by

$$\text{Re} = \frac{\rho v D}{\mu} \quad (4)$$

where ρ and μ are the density and dynamic viscosity of the air, which are 1.225 kg/m^3 and 1.825x10^{-5} Pa-s, respectively; υ is the velocity of the air where in Rochester, NY, the wind speed is approximately 3.13-4.47 m/s based on averaged data from 1991-2020 [8]; D is the rotor diameter of the reference turbine, which is approximately 0.94 m. This corresponds to Re = 197490-282039. To maintain dynamic similarity the Re of the tested turbines must be within a factor of 4 of the reference turbine to ensure scalability. The tested wind speeds ranged from 4.25-9.68 m/s. The tested density and dynamic viscosity of the air were approximately 1.192 kg/m^3 and 1.839x10^{-5} Pa-s respectively. The measured diameter of the printed curved and toroidal models were 0.35402 m and 0.35422 m, respectively. Using Equation 4, this corresponds to the curved design having Re = 97524-222125 and the toroidal design having Re = 97579-222251.

To better compare acoustic performance of each blade relative to its aerodynamic effectiveness, a normalized sound metric was introduced. The RMS pressure derived from SPL was normalized by the electrical power output per unit volumetric flow rate, given by

$$\text{Normalized Sound Metric} = \frac{P_{RMS}}{\frac{V^2}{R}\frac{1}{vA}} \quad (5)$$

Where P_{RMS} is the RMS sound pressure obtained from the SPL measurements, V is the measured generator voltage across the load resistor, R is the load resistance, v is the measured wind speed, and A is the swept area of the turbine blades. This normalization compares acoustic output relative to useful electrical power generation and airflow through the turbine.

The method to obtain the isolated blade SPL was through time-domain background subtraction. For each wind speed condition, multiple trials were conducted, as aforementioned. The background signals were averaged across trials to create a motor noise signature for each RPM condition. By subtracting the time-domain signal of the blade running with the motor to the motor

noise signature, the isolated blade signal is obtained. The RMS of the isolated blade signal is used in Equation 1 to get the SPL. Appendix E shows the MATLAB code used for analyzing sound data. Appendix D describes how the error bars for each graph were made.

4. Results

The experiment results are presented as the efficiency versus wind speed, the TSR versus wind speed, isolated blade SPL versus wind speed, blade SPL versus rotor speed, and normalized sound versus wind speed.

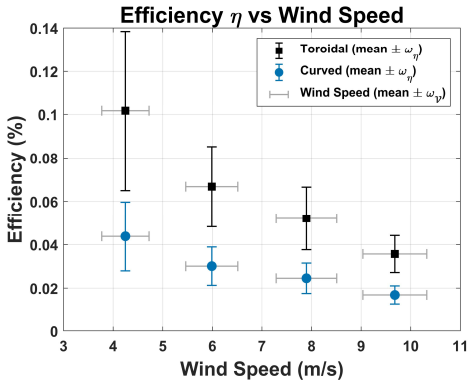


Figure 6: The power efficiency versus measured wind speed for both geometries with error bars. The toroidal blades were significantly more efficient at all wind speeds.

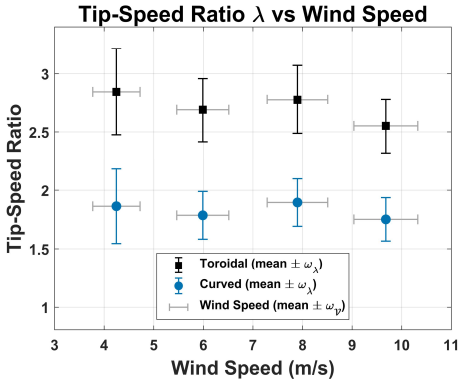


Figure 7: The TSR of each blade vs measured wind speed with error bars. The toroidal blade had a consistently higher TSR compared to the curved blades.

Figure 6 goes against the initial hypothesis by showing that the toroidal blades were more efficient than the curved blades at all tested wind speeds. The toroidal blades also had consistently higher TSR values, as shown in Figure 7, indicating that they operated at a higher rotational speed relative to the incoming wind. These results suggest that the toroidal geometry improved energy generation in this setup, though further testing would be needed to determine whether the same trend persists at larger scale or under optimized loading conditions.

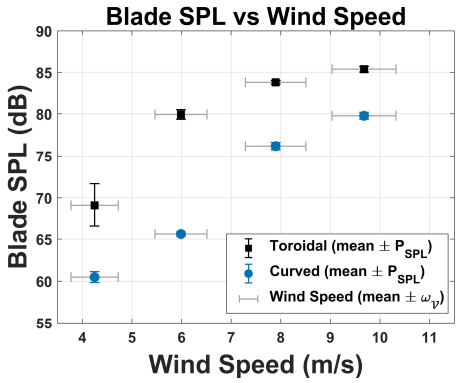


Figure 8: The isolated blade SPL for each blade geometry using time-domain subtraction of the motor background signal from the blade and background signal. Error bars show a random uncertainty at 95% confidence using the t-distribution.

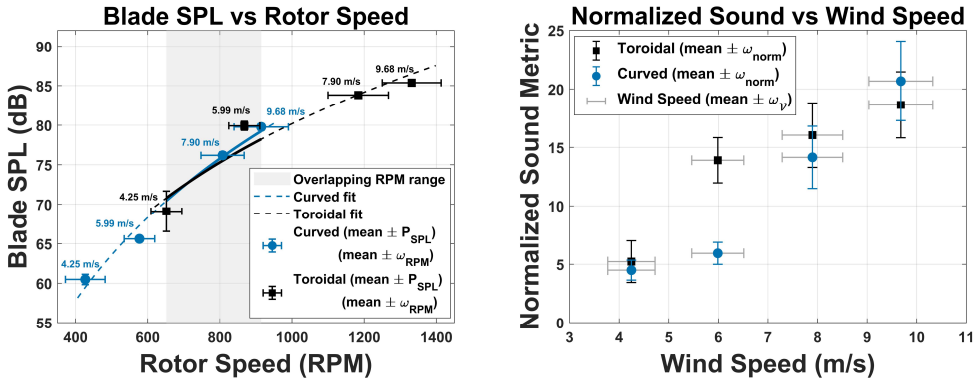


Figure 9: (Left): Blade SPL versus rotor speed with the overlapping RPM range of 652-915 RPM with error bars. (Right): Normalized sound metric versus wind speed with error bars, where RMS pressure is normalized by electrical power output per unit volumetric airflow.

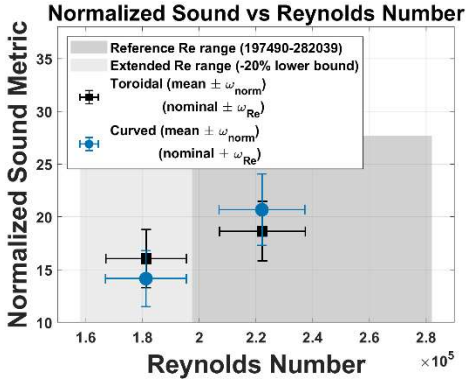


Figure 10: Normalized sound metric versus Reynolds number with error bars.

Figure 8 shows an expected relationship between wind speed and blade SPL, as both parameters increase consistently. Across all four wind speeds, the toroidal blade was louder than the curved by 5.5-8.7 dB. This difference in SPL can be explained by the RPM of the toroidal blade, which was consistently ~50% faster than the curved blade for a given wind speed. These results suggest that the toroidal geometry does not offer a noise advantage over the curved blade design at the same input wind speed because RPM has a stronger influence on noise output than blade shape

alone. Therefore, greater emphasis is placed on Figure 9, where RPM is controlled and RMS pressure derived from SPL is normalized by power per unit volumetric airflow. Within the overlapping RPM range, the toroidal blade ranges from 0.4 dB louder at lower RPMs to 1.0 dB quieter at higher RPMs. The logarithmic trends suggest that at higher wind speeds, the two curves eventually cross with the toroidal becoming quieter. The extrapolated trend lines suggest that at RPMs above the overlap range, the toroidal blades are consistently quieter than the curved blade, suggesting that the toroidal blade noise advantage emerges at higher RPM. When RMS pressure is normalized by power per unit volumetric airflow, the toroidal blade shows an approximately 10% lower normalized sound metric than the curved blade at 9.68 m/s; however, the overlapping uncertainty bars suggest this should be interpreted as a trend rather than a definitive difference. At higher wind speeds, the toroidal blade may produce lower relative noise because its geometry could promote smoother airflow, as seen in related marine applications. A deviation throughout the noise trials was the 5.99 m/s trial for the curved blades, which had a noticeably lower SPL in Figure 8 and 9. This was likely because this particular trial was tested in a different testing setup in a noisier environment that may have introduced echo and additional variability to the audio file.

Figure 10 adds scale context to the normalized sound comparison by plotting the same metric against the Reynolds number used for the whole-rotor analysis. The lower speed points fall near the extended Reynolds range, while the higher speed points fall within the reference turbine range, showing that the acoustic trend is being evaluated under flow conditions reasonably close to the intended application. This complies with the right side of Figure 9, where the toroidal blade has a slightly lower normalized sound metric at the highest tested condition. However, because the uncertainty bars still overlap, Figure 10 should be treated as supporting evidence of a possible high speed acoustic advantage rather than proof of a definitive difference.

Together, these results demonstrate that the toroidal blade shows strong power performance and competitive acoustic behavior when compared at similar rotor speeds and when normalized by power output per unit airflow, suggesting that its marine-inspired geometry may translate favorably to wind turbine applications for residential areas.

5. Discussion

The power testing showed that the toroidal blades were more efficient and had a higher TSR than the curved blades at all tested wind speeds, suggesting that the toroidal design was better at generating energy in this setup. The acoustic results suggest that the toroidal blades had competitive sound performance over the overlapping RPM range, and the normalized sound metric indicated a possible acoustic advantage at the highest tested wind speed. This implies that toroidal blades may be a viable option for residential applications in Rochester, especially when acoustic output is considered relative to power generation.

One limitation with the results is that the blade designs were not equal. Although they both had a NACA 0018 airfoil, the same material, the same rotor diameter, and almost the same average chord length, their surface areas were largely different. This was an obstacle when outdoor testing was attempted, because the curved blades could not spin as well as the toroidal ones due to how much smaller its surface area was, which prevented it from overcoming the internal resistance of the motor. In that same regard, although testing indoors allowed for a more controlled environment where the wind speed could be the same for every trial, it was not reflective of a real setting in which wind speed is variable.

There are several aspects of this project that future experiments could improve upon. If one were to take a toroidal blade and straighten it, it would essentially be double the length of a curved blade. Hence, in terms of manufacturing, it could be a fairer comparison if the current toroidal blade

design was instead evaluated against a six-bladed curved design. This increase in surface area would also make the blade more likely to spin outdoors, and the greater mass would make the MOI closer to the toroidal blades' value. Most curved blades on real wind turbines employ different airfoils along the blade's length to maximize lift (whereas in this experiment both designs always used the same airfoil). To make the comparison more accurate, one could replace the current curved design with a more conventional design that uses a variable airfoil. Post-processing wise, applying an epoxy layer to fill the voids in the prints could improve performance.

As for testing, vibrations were not fully damped in the setup, which could have interfered with RPM and noise measurements. Future work could involve recreating the tower with PVC, which is sturdier than wood. For stronger noise testing results, a pre-calibrated microphone would substantiate the findings to a standardized, quantitative acoustic measurement. The FIFINE microphone used in the experiment was not calibrated to an absolute pressure reference; therefore, SPL values are relative to a normalized reference rather than dB A-weighted (human hearing). While the study remains comparative between the two geometries, direct comparison with published, standardized literature cannot be fully made. Future experiments should prioritize using a calibrated microphone or calibrating the microphone with a pistonphone.

Finally, the scope of this project was limited to energy and sound performance. However, cost is also a crucial comparison in the construction of wind turbines. The toroidal blades are undoubtedly more complex than curved blades and would cost more to manufacture and implement. Whether the toroidal blades' benefits outweigh the added cost in residential applications is a valid subject for future study.

6. Acknowledgements

We would like to thank our teaching assistant Kenneth "CJ" Thompson and Professor Ibrahim Mohammad for their expertise and guidance throughout this project, Chris Pratt and Jim Alkins for providing the leaf blower, and the Rettner Help Desk staff Marcos Ibáñez Matles, Jack Stampone, Steven Wang, and Enrique Gonzalez for their 3D printing consultations for the turbine blades.

References

[1] Kramarchuk, R., and d'Avack, F., 2025, "S&P Global Energy Horizons Top Trends 2026," S&P Global, https://www.spglobal.com/energy/en/news-research/special-reports/energy-transition/horizons-top-cleantech-trends-2026.

[2] Zhu, F., Li, B., Dai, and Zhou, W., 2025, "Study on the Flow Field Characteristics of Turbine Blades of Roof-Type Wind Turbines with Different Shapes of Wind Aggregation," Proceedings of the ICMTIM, Nanjing, China, April 11-13, 2025, pp. 341-345. 10.1109/ICMTIM65484.2025.11041423.

[3] Pawlaczyk-Łuszczyńska, M., Zaborowski, K., Dudarewicz, A., Zamojska-Daniszewska, M., and Waszkowska, M., 2018, "Response to Noise Emitted by Wind Farms in People Living in Nearby Areas," *International Journal of Environmental Research and Public Health,* 15(8), p. 1575. https://doi.org/10.3390/ijerph15081575.

[4] Zheng, Z., and Chen, L., 2024, "Aerodynamic Performance and Wake Development of NACA 0018 Airfoil With Serrated Gurney Flaps," *Frontiers in Energy Research,* 12. https://doi.org/10.3389/fenrg.2024.1363402.

[5] Bai, J., Li, Y., Liu, X., Zhang, H., and Ren, L., 2025, "Hydrodynamic Performance and Vortex Structure Analysis of a Toroidal Propeller," *Journal of Marine Science and Engineering,* 13(6), p. 1046. https://doi.org/10.3390/jmse13061046.

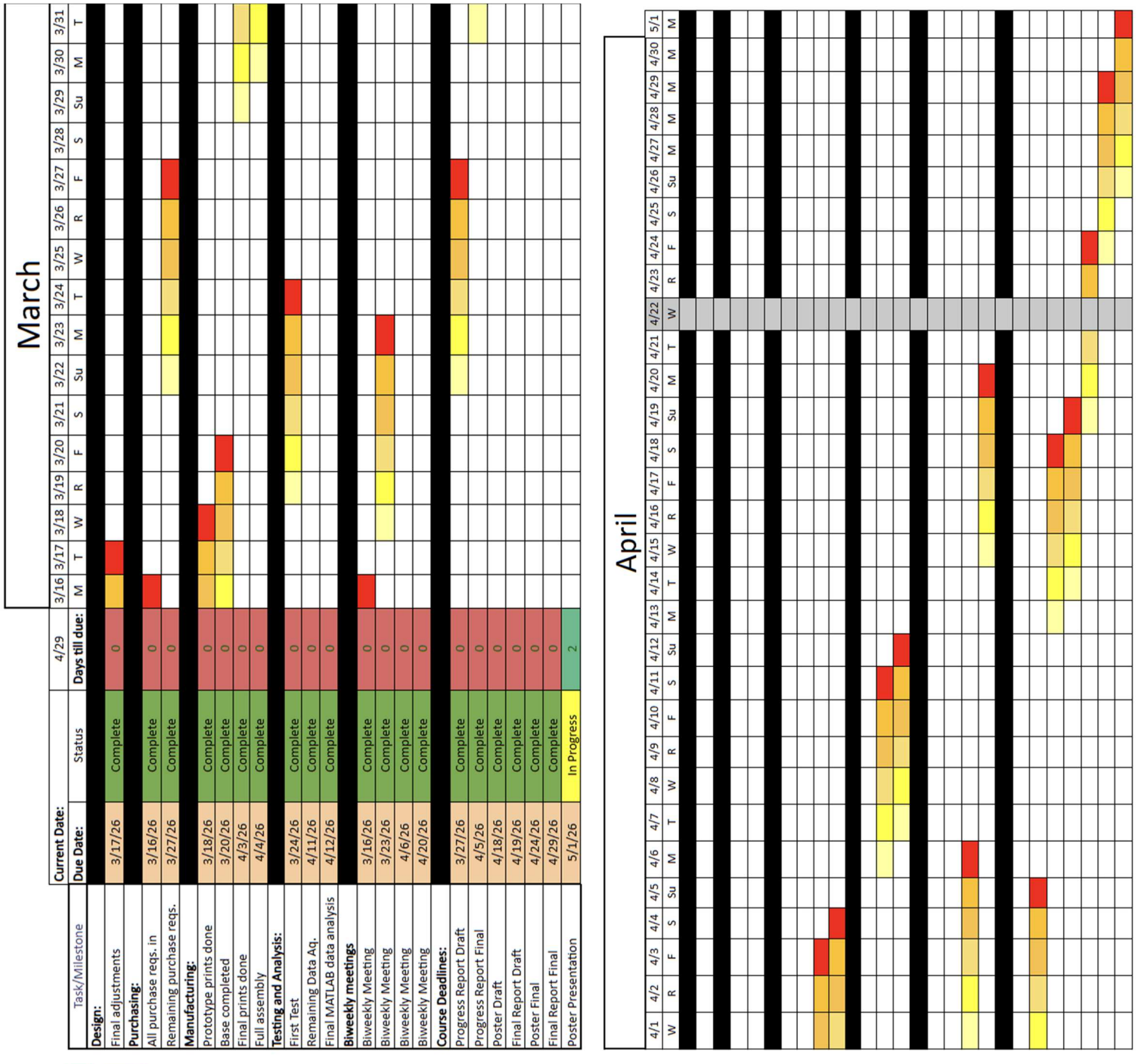
[6] Raghavendra, S., Ravikumar, T. R., Reddy, G. Y., Manjunatha, K. N., and Madhusudhana, S. V., 2020, "Design of Wind Blades for the Development of Low-Power Wind Turbines Using Betz and Schmitz Methods," *Advances in Materials and Processing Technologies*, 8(1), pp. 808-827. https://doi.org/10.1080/2374068X.2020.1833605.

[7] "Anemometer Wind Speed Sensor w/Analog Voltage Output," Adafruit Industries, https://www.adafruit.com/product/1733.

[8] Kator, D., Casey N.W., Menne, M.J., Buddenberg, A., 2023, "Local Climatological Data (LCD), Version 2, Normals, Means, and Extremes Rochester (KROC)," NOAA National Centers for Environmental Information, https://www.ncei.noaa.gov/access/metadata/landing-page/bin/iso?id=gov.noaa.ncdc:C01689.

Appendix A: Project Timeline

Table 3: Gantt Chart for the project. The left image shows the first half of the chart, and the right image shows the right half of the chart. The dates span from March 16 to May 1.



Appendix B: 2D CAD Drawing Package

Table 2: Table referencing the chord lengths of the NACA 0018 airfoil profiles used to create the turbine geometries at each section shown in their 2D CAD drawings.

	Section Line A	Section Line B	Section Line C
Curved Blade Chord Length (mm)	15.27	24.67	43.47
Curved Section Distance to Datum A	156.55	82.53	8.51
Toroidal Blade Chord Length (mm)	28.97	28.97	28.97
Toroidal Section Distance to Datum A	163.91	61.47	0

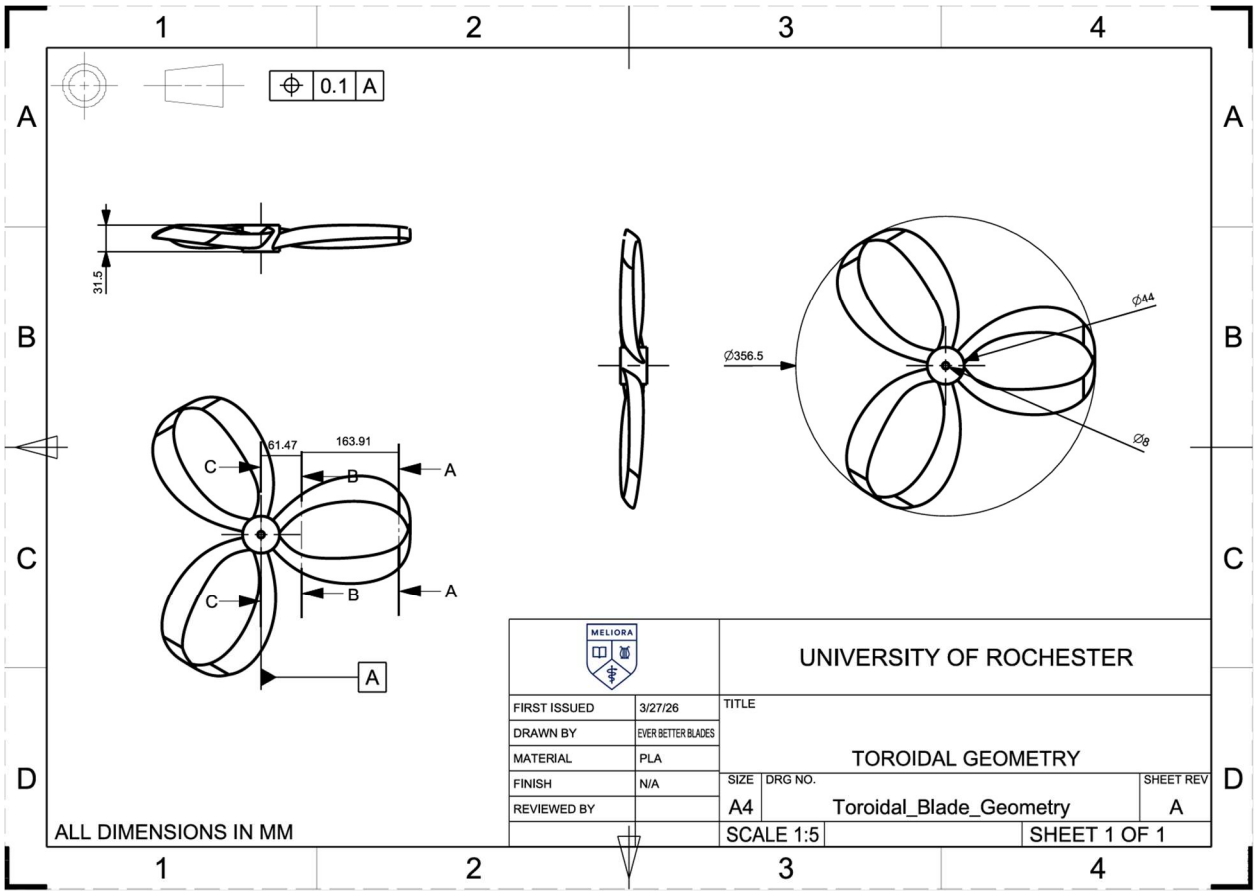


Figure 11: 2D CAD drawing of the toroidal blade design. Sections show the locations of the NACA 0018 airfoil cross-sections. These sections were connected using a 3D fit spline and are described in Table 2. All dimensions are in mm.

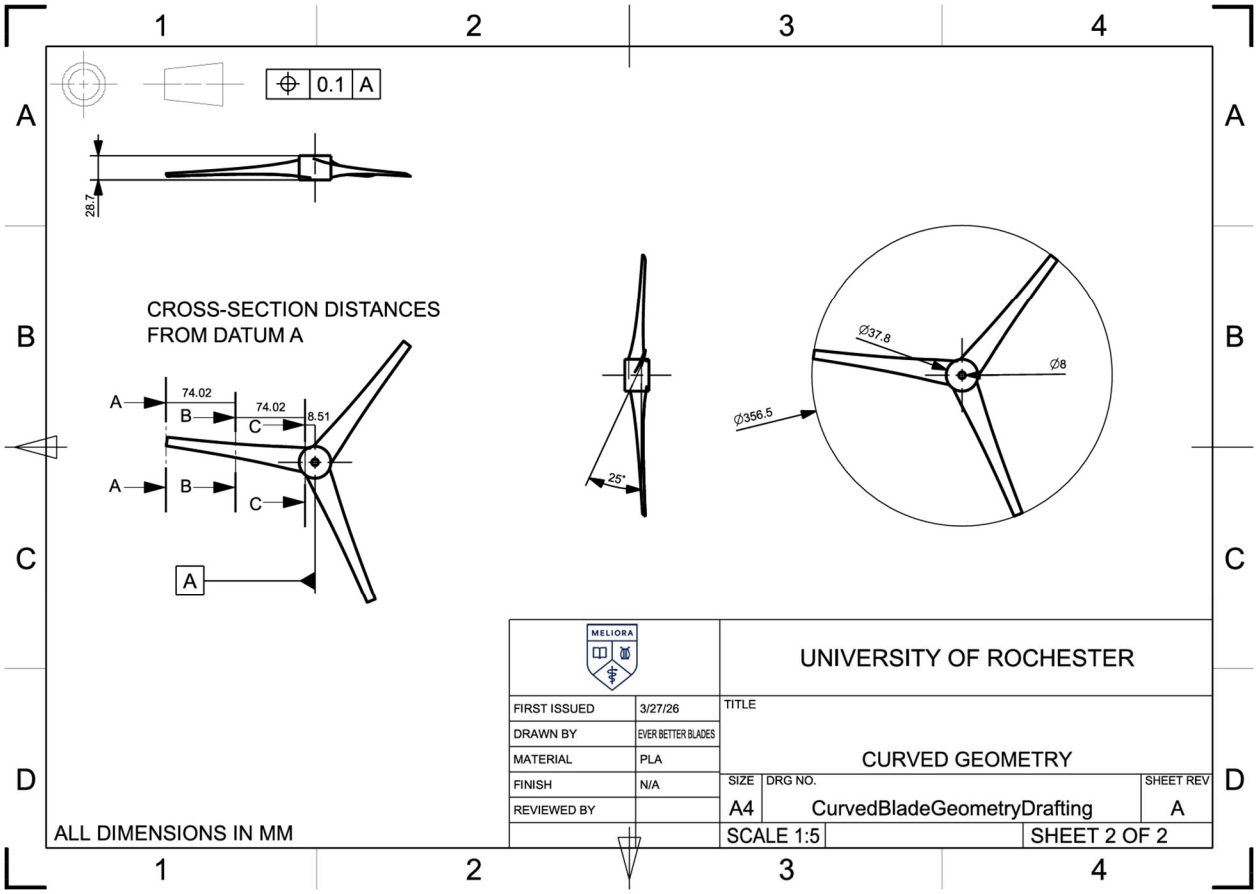
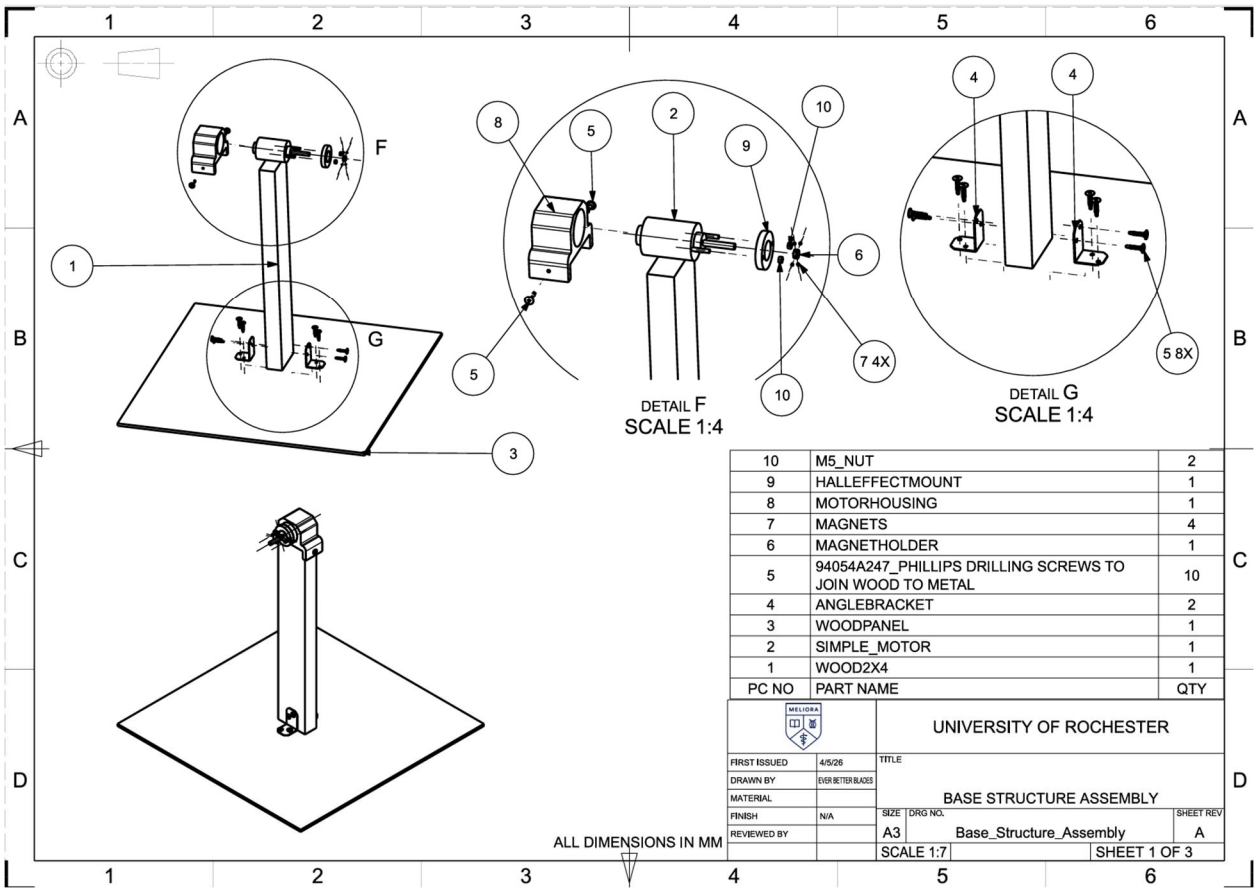


Figure 12: 2D CAD drawing of the curved blade design. Sections show the locations of the different NACA 0018 airfoil cross-sections. These sections are described in detail in Table 2. All dimensions are in mm.



PC NO	PART NAME	QTY
10	M5_NUT	2
9	HALLEFFECTMOUNT	1
8	MOTORHOUSING	1
7	MAGNETS	4
6	MAGNETHOLDER	1
5	94054A247_PHILLIPS DRILLING SCREWS TO JOIN WOOD TO METAL	10
4	ANGLEBRACKET	2
3	WOODPANEL	1
2	SIMPLE_MOTOR	1
1	WOOD2X4	1

Figure 13: 2D CAD assembly drawing of the base structure design. A parts list is included.

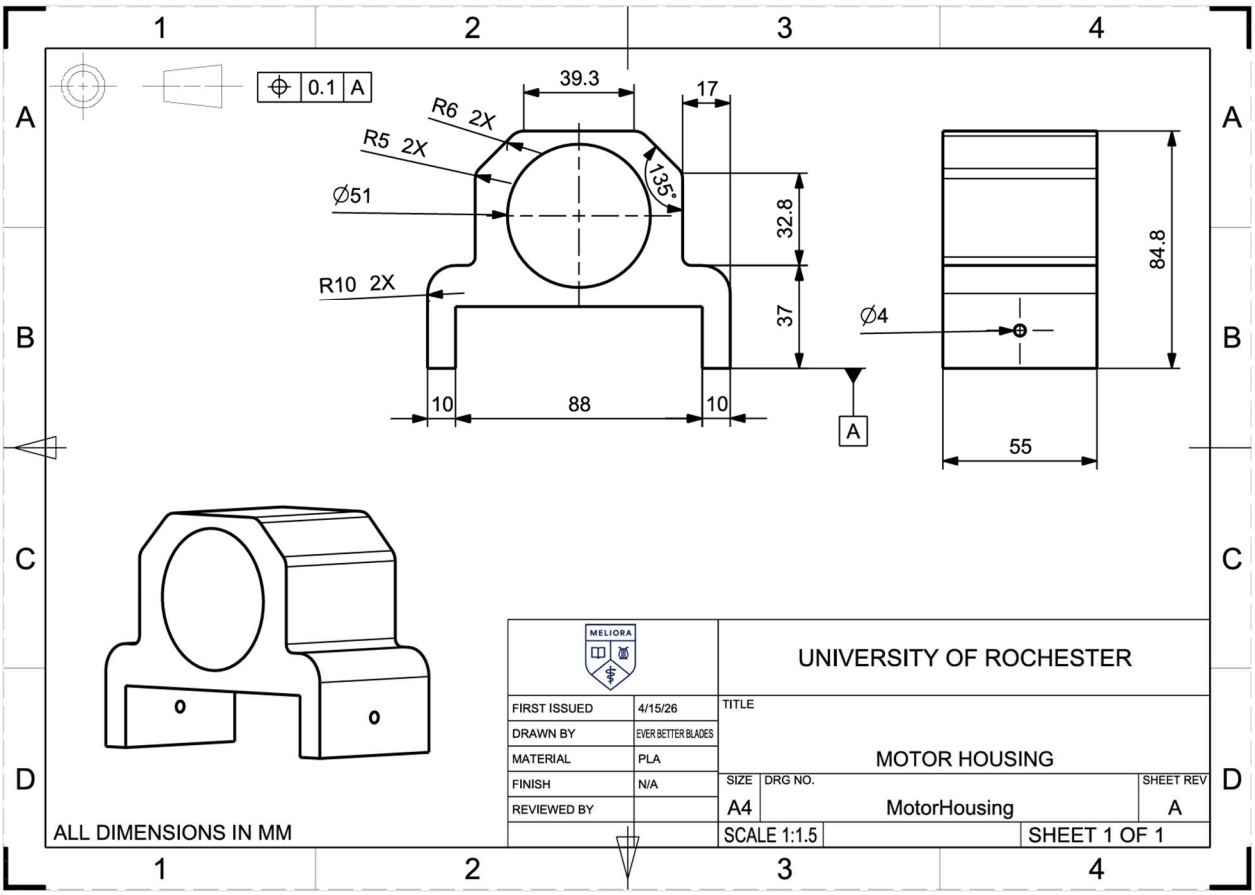


Figure 14: 2D CAD drawing of the motor housing unit. All dimensions are in mm.

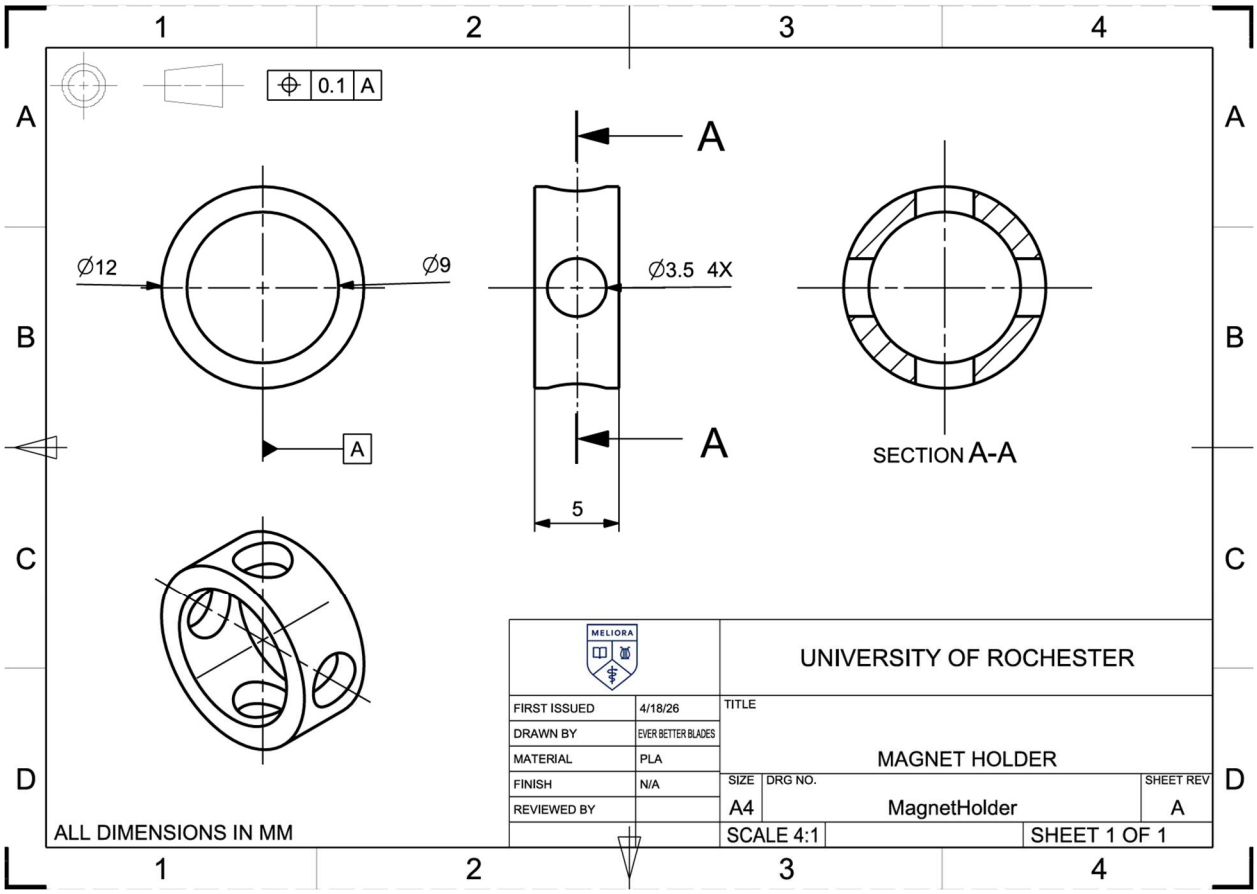


Figure 15: 2D CAD drawing of the magnet holder. All dimensions are in mm.

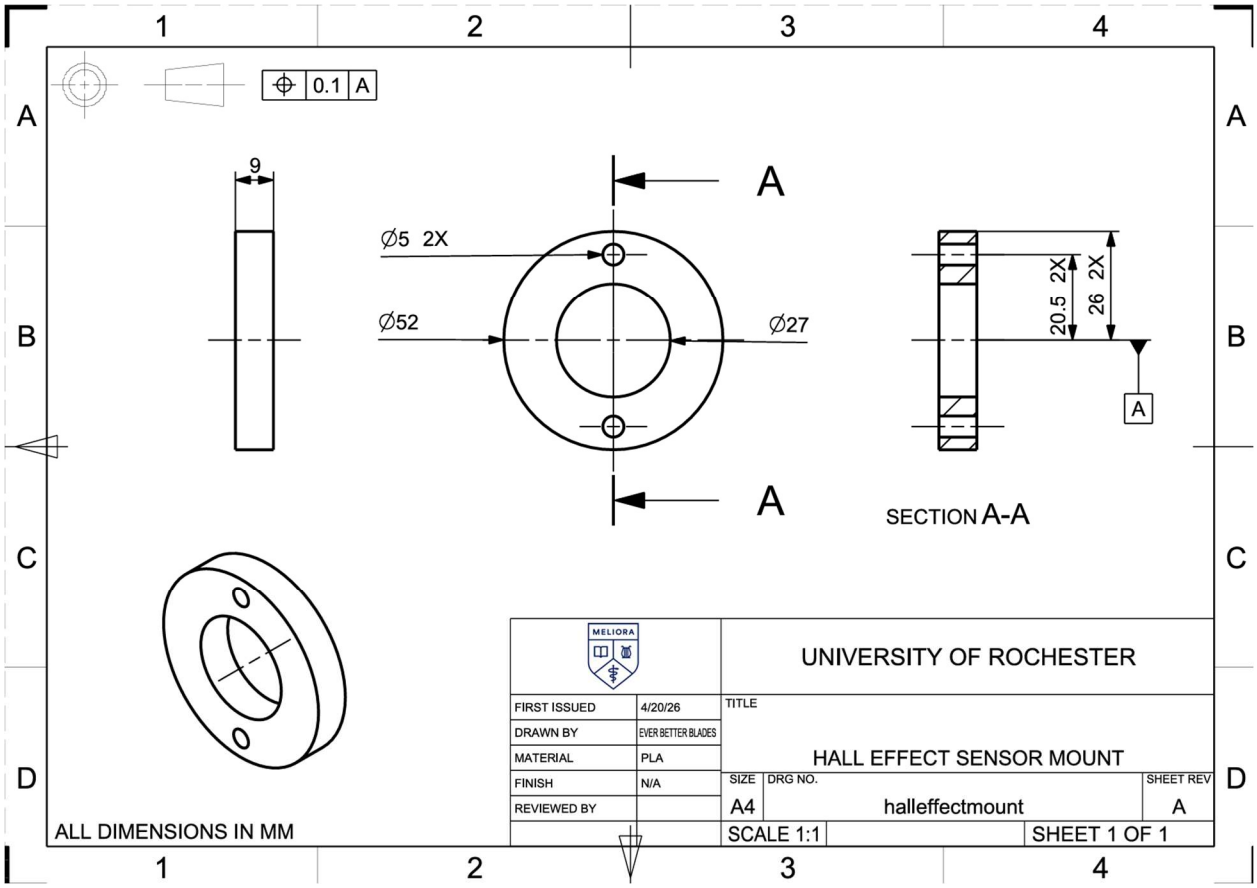


Figure 16: 2D CAD drawing of the Hall effect sensor mount. All dimensions are in mm.

Appendix C: Bill of Materials

Quantity	Description	Vendor	Order Number	Price
2	High Torque DC Motor 12V/24V	Amazon	2603021523	42.18
1	Anemometer Wind Speed Sensor	Adafruit	2603021549	44.95
3	Hall Effect Sensor - US5881LUA	Adafruit	2603021549	6.00
1	ELEGOO 6PCS 400 Point Breadboard Kit	Amazon	2603092113	8.99
1	120PCS Breadboard Jumper Wires 20cm	Amazon	2603092113	6.98
1	Solid Rubber Sheet Roll 1/8” Thick	Amazon	2603092113	9.54
2	FIFINE USB Microphone	Amazon	2603092113	59.98
1	TRYMAG Tiny 3x2mm 280PCS	Amazon	2603092113	7.99
1	Uxcell 6PCS 8mm Shaft Lock Collar	Amazon	2603092113	8.29
2	2”x4”x8’ Wood	Home Depot	2603092219	7.70
1	J-B Weld Original Cold-Weld™	Home Depot	2603092219	7.68
4	Angle Bracket	Home Depot	2603092219	5.28
2	2x2 Wood Sheet	Home Depot	2603092219	17.26
1	Phillips #10 Screws	McMaster	2603092241	16.65
2	Rubber P Clamp	McMaster	2603092241	9.28
1	Arduino Uno REV3	Amazon	2603231156	27.60
1	Initial Curved Blades 3D Print	Rettner	N/A	1.51
3	Single Toroidal Blade Prototype 3D Print	Rettner	N/A	0.47
1	Toroidal Blades Final 3D Print	Rettner	N/A	4.90
1	Curved Blades Final 3D Print	Rettner	N/A	2.21
1	Leaf Blower, Batteries, Charger	Hopeman	N/A	0.00
1	Sandpaper	Hopeman	N/A	0.00
2	Magnet Holder 3D Prints	Hopeman	N/A	0.00
2	Motor Housing 3D Prints	Hopeman	N/A	0.00
	Shipping			13.78
		Total Cost		**309.22**

Appendix D: Uncertainty Analysis

In general, the uncertainty of a result R as a function of n variables ($x_{1...n}$) can be found using a root sum square (RSS):

$$\omega_R = \left(\sum_{i=1}^{n} \left(\frac{\delta R}{\delta x_i} \omega_{X_i} \right)^2 \right)^{\frac{1}{2}} \tag{6}$$

where ω is the uncertainty of a specified variable. However, if the equation for R has the form of

$$R = C x_1^a x_2^b \dots x_n^n \tag{7}$$

where C is a constant, and $a, b, \dots, n$ are their exponents, then Equation 6 can be simplified to

$$\frac{\omega_R}{R} = \left[\left(\frac{a\omega_1}{x_1} \right)^2 + \left(\frac{b\omega_2}{x_2} \right)^2 + \dots + \left(\frac{n\omega_n}{x_n} \right)^2 \right]^{\frac{1}{2}} \tag{8}$$

Additionally, there are two types of uncertainty: systematic and random. The total systematic uncertainty for a single measurement can be obtained by taking the RSS of all of the systematic uncertainties. On the other hand, random uncertainty for a variable x can be calculated with

$$P_X = tS_X \tag{9}$$

where t is the t-value (a function of the degrees of freedom and the confidence level) and S_X is the variable's standard deviation. The total uncertainty in a measurement can be obtained by taking the RSS of the total systematic uncertainty and the random uncertainty.

Since all of the equations of interest in this paper (power efficiency, tip-speed ratio, and SPL) are in the form of Equation 7, Equation 8 was used to create the error bars ($\pm \omega$) in each measurement corresponding to a certain wind speed.

The best way to visualize this entire process is by referencing the analysis code in Appendix E. To summarize the creation of the code, a list of all of the systematic errors in each measurement device (caliper, thermocouple, multimeter, anemometer, Arduino) used in this project was created mainly by looking through their specification sheets. All of the types of errors that were found were: resolution, accuracy, and quantization, and they were put into an RSS to get the total systematic uncertainty for their respective measurements (such as resistance, swept area etc.). Then the standard deviation in voltage output, RPM, and isolated blade SPL at each wind speed (recalling that every test at a certain wind speed had four trials) was calculated. The standard deviation in wind speeds recorded on the anemometer was calculated as well. These standard deviations were plugged into Equation 9 to get the random uncertainties for those four variables. Finally, an RSS of the total systematic uncertainty and random uncertainty was taken, and the total uncertainty for each variable was plugged into Equation 8 to get uncertainty (and consequently the error bars) for a particular value of interest. One exception to note is that SPL data did not have any systematic uncertainty, because the microphone did not come with a datasheet, so only random uncertainty was used to make its error bars.

The only measurements that did not have an error were the pressure and gas constant of air, which was used to find its density (a variable used in Equation 2). The density of the air and its gas constant in Hopeman was assumed to be 1 atm (101,325 Pa) and 287.05 J/ kg°K respectively. Air was considered to be an ideal gas, and its density ρ was calculated utilizing the modified ideal gas law

$$\rho = \frac{P}{RT} \tag{10}$$

where R is the gas constant of air, T is the temperature of the air, and P is the pressure of the air (equivalent to the room's pressure). Likewise, Equation 8 was used on Equation 10 to calculate the error in the density of air.

One final note for the code is that the resolution error in the RPM measurements was found to be 30 RPM. To elaborate, every shaft had 4 magnets attached to it. Whenever a magnet passes by the Hall sensor, it increases the count in the Arduino data collection code by 1 (see Appendix F). That means that 4 counts in the Arduino code are equivalent to 1 revolution in real life. Arduino calculated the number of counts over each second long interval. To convert this to RPM, the count values had to be multiplied by 60 seconds per minute and divided by 4 counts per rotation. This means the resolution in the code is 15 RPM. However, physically if the Hall sensor starts taking data at the falling edge of a magnet and ends at the rising edge of a magnet, that gives it a worst-case resolution error of 30 RPM, which was the value used for the RPM error bars.

Appendix E: MATLAB Analysis Code

1. VoltWindRPM.m, used to analyze voltage output, wind speed, and RPM data. Created Figures 6 and 7

```
%% Voltage output, wind speed, and RPM processing & analysis script
% VoltWindRPM, written 19 April 2026 by Jose Gutierrez, Aidan Sabin, and Aaron Tran
% Before using this script, import all matrices (Hall sensor counts, ADC values)
close all
%% 4.25 m/s toroidal
% 4 magnets equals 1 rotation and each data point in the array is over a 1 second time interval.
% Therefore, multiplying hall sensor counts by 15 gives the RPM.
% The ADC ranges from 0-5 V. 0 V = 0 ADC and 5 V = 1023 ADC.
% Therefore, multiplying by 5 / 1023 gives the voltage output value.
t425mean_rpm = [mean(h_t425_1), mean(h_t425_2), mean(h_t425_3), mean(h_t425_4)] * 15;
% ^ Xspeedmean_rpm is a 1x4 matrix with the average RPM per trial

t425_rpm = mean([h_t425_1 h_t425_2 h_t425_3 h_t425_4] * 15);
% ^ Average RPM across all trials

tmean_voltages_425 = [mean(adc_t425_1), mean(adc_t425_2), mean(adc_t425_3), mean(adc_t425_4)] * 5 / 1023;
% ^ Xmean_voltages_speed is a 1x4 matrix with the average voltage per trial

V_out_t425 = mean([adc_t425_1 adc_t425_2 adc_t425_3 adc_t425_4] * 5 / 1023);
% ^ Average voltage across all trials
% fprintf('4.25 m/s Toroidal\nAverage rpm: %0.2f\n', t425_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_t425)
%% 4.25 m/s curved
c425mean_rpm = [mean(h_c425_1), mean(h_c425_2), mean(h_c425_3), mean(h_c425_4)] * 15;
c425_rpm = mean([h_c425_1 h_c425_2 h_c425_3 h_c425_4] * 15);
cmean_voltages_425 = [mean(adc_c425_1), mean(adc_c425_2), mean(adc_c425_3), mean(adc_c425_4)] * 5 / 1023;
V_out_c425 = mean([adc_c425_1 adc_c425_2 adc_c425_3 adc_c425_4] * 5 / 1023);
% fprintf('4.25 m/s Curved\nAverage rpm: %0.2f\n', c425_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_c425)
%% 5.99 m/s toroidal
t599mean_rpm = [mean(h_t599_1), mean(h_t599_2), mean(h_t599_3), mean(h_t599_4)] * 15;
t599_rpm = mean([h_t599_1 h_t599_2 h_t599_3 h_t599_4] * 15);
tmean_voltages_599 = [mean(adc_t599_1), mean(adc_t599_2), mean(adc_t599_3), mean(adc_t599_4)] * 5 / 1023;
V_out_t599 = mean([adc_t599_1 adc_t599_2 adc_t599_3 adc_t599_4] * 5 / 1023);
% fprintf('5.99 m/s Toroidal\nAverage rpm: %0.2f\n', t599_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_t599)
%% 5.99 m/s curved
c599mean_rpm = [mean(h_c599_1), mean(h_c599_2), mean(h_c599_3), mean(h_c599_4)] * 15;
c599_rpm = mean([h_c599_1 h_c599_2 h_c599_3 h_c599_4] * 15);
cmean_voltages_599 = [mean(adc_c599_1), mean(adc_c599_2), mean(adc_c599_3), mean(adc_c599_4)] * 5 / 1023;
V_out_c599 = mean([adc_c599_1 adc_c599_2 adc_c599_3 adc_c599_4] * 5 / 1023);
% fprintf('5.99 m/s Curved\nAverage rpm: %0.2f\n', c599_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_c599)
%% 7.90 m/s toroidal
t790mean_rpm = [mean(h_t790_1), mean(h_t790_2), mean(h_t790_3), mean(h_t790_4)] * 15;
t790_rpm = mean([h_t790_1 h_t790_2 h_t790_3 h_t790_4] * 15);
tmean_voltages_790 = [mean(adc_t790_1), mean(adc_t790_2), mean(adc_t790_3), mean(adc_t790_4)] * 5 / 1023;
V_out_t790 = mean([adc_t790_1 adc_t790_2 adc_t790_3 adc_t790_4] * 5 / 1023);
% fprintf('7.90 m/s Toroidal\nAverage rpm: %0.2f\n', t790_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_t790)
%% 7.90 m/s curved
c790mean_rpm = [mean(h_c790_1), mean(h_c790_2), mean(h_c790_3), mean(h_c790_4)] * 15;
c790_rpm = mean([h_c790_1 h_c790_2 h_c790_3 h_c790_4] * 15);
cmean_voltages_790 = [mean(adc_c790_1), mean(adc_c790_2), mean(adc_c790_3), mean(adc_c790_4)] * 5 / 1023;
V_out_c790 = mean([adc_c790_1 adc_c790_2 adc_c790_3 adc_c790_4] * 5 / 1023);
% fprintf('7.90 m/s Curved\nAverage rpm: %0.2f\n', c790_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_c790)
%% 9.68 m/s toroidal
t968mean_rpm = [mean(h_t968_1), mean(h_t968_2), mean(h_t968_3), mean(h_t968_4)] * 15;
t968_rpm = mean([h_t968_1 h_t968_2 h_t968_3 h_t968_4] * 15);
tmean_voltages_968 = [mean(adc_t968_1), mean(adc_t968_2), mean(adc_t968_3), mean(adc_t968_4)] * 5 / 1023;
V_out_t968 = mean([adc_t968_1 adc_t968_2 adc_t968_3 adc_t968_4] * 5 / 1023);
% fprintf('9.68 m/s Toroidal\nAverage rpm: %0.2f\n', t968_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_t968)
%% 9.68 m/s curved
c968mean_rpm = [mean(h_c968_1), mean(h_c968_2), mean(h_c968_3), mean(h_c968_4)] * 15;
c968_rpm = mean([h_c968_1 h_c968_2 h_c968_3 h_c968_4] * 15);
cmean_voltages_968 = [mean(adc_c968_1), mean(adc_c968_2), mean(adc_c968_3), mean(adc_c968_4)] * 5 / 1023;
V_out_c968 = mean([adc_c968_1 adc_c968_2 adc_c968_3 adc_c968_4] * 5 / 1023);
% fprintf('9.68 m/s Curved\nAverage rpm: %0.2f\n', c968_rpm)
% fprintf('Average Voltage Output: %0.2f\n\n', V_out_c968)
%% Comparing designs
rawvoltages = [0.615, 0.608, 0.605, 0.611; % Wind speed 1
    0.697, 0.695, 0.700, 0.691;  % Wind speed 2
    0.799, 0.783, 0.791, 0.787;  % Wind speed 3
```

```
    0.877, 0.884, 0.872, 0.879]; % Wind speed 4
all_wind_speeds = 20.25 * rawvoltages - 8.1; % Convert to wind speed with linear scale
% ^ Each row is a wind speed, each column is a trial
anemometer_readings = mean(rawvoltages,2)'; % Each column is a speed
wind_speeds = 20.25 * anemometer_readings - 8.1; % averaged wind speeds converted to calculated using linear scale
% toroidal averages, each column is a speed
rpm_t = [t425_rpm t599_rpm t790_rpm t968_rpm];
Vout_t = [V_out_t425 V_out_t599 V_out_t790 V_out_t968];
% curved averages
rpm_c = [c425_rpm c599_rpm c790_rpm c968_rpm];
Vout_c = [V_out_c425 V_out_c599 V_out_c790 V_out_c968];
%% Efficiency Calculation
temperature_C = 23; % Temperature of Hopeman, measured using a K-type
                    % thermocouple attached to an AstroAI DM6000AR multimeter
Btemp1 = 1; % °C, Resolution error
Btemp2 = temperature_C * 0.01 + 1*4; % Accuracy of ± (1.0% + 4) for Celsius
wtemp = rssq([Btemp1, Btemp2]); % Uncertainty in the temperature
temperature_K = temperature_C + 273.15; % converting to Kelvin
% This conversion doesn't affect Btemp2 though because one degree C
% is equal to the size of one degree K
atmospheric_pressure = 101325; % Pascals, pressure inside Hopeman assuming 1 atm
R_air = 287.05; % J/(kg*K), gas constant of air via NASA Planetary Data System

rho = atmospheric_pressure / (R_air * temperature_K); % air density in kg/m^3
wrho = rho*(rssq(wtemp/temperature_K)); % Putting the modified ideal gas law in the
% form of Equation X2 and taking P and R to be errorless to get uncertainty of density

radC = 177.01 / 1000; % meters
radT = 177.11 / 1000; % meters
Brad1 = 0.01/1000; % mm, Resolution error of 0.01 mm converted to m
Brad2 = 0.03/1000; % Accuracy of ± 0.03 mm converted to m
wradC = rssq([Brad1, Brad2]); wradT = wradC; % Uncertainties are same for both
% ^ Uncertainties in radii

% Swept areas of both designs
areaC = pi * radC ^ 2; % meters^2
areaT = pi * radT ^ 2; % meters^2
wareaC = areaC*rssq(2*wradC/radC); wareaT = areaT*rssq(2*wradT/radT);
% ^ Uncertainties in areas

% Load resistance across the output of the generator
R_load = 1000; % ohms
Br1 = 0.001; % Resolution error
Br2 = R_load * 0.015 + 1*3; % Accuracy of ± (1.5% + 3) ohms
wr = rssq([Br1, Br2]); % Uncertainty in resistance

% Quantization error from Arduino is ± 0.5*(Vru - Vrl) / 2^N where Vru is
% the upper voltage, Vrl is the lower voltage, and N is the bit depth
Bv = 0.5*((5-0)/2^10);
% Random uncertainty in a single measurement is t*s
% where s is the standard deviation and t is the t-value
tvoltdev(1) = std(tmean_voltages_425); tvoltdev(2) = std(tmean_voltages_599);
tvoltdev(3) = std(tmean_voltages_790); tvoltdev(4) = std(tmean_voltages_968);
cvoltdev(1) = std(cmean_voltages_425); cvoltdev(2) = std(cmean_voltages_599);
cvoltdev(3) = std(cmean_voltages_790); cvoltdev(4) = std(cmean_voltages_968);
% Degrees of freedom v = n-1, so v = 3. a = 1 - confidence level, so 0.05 for
% a confidence of 95%. a/2 = 0.025. Use v and a/2 for Table 6.6 on Page 158 in
% Introduction to Engineering Experimentation by Wheeler and Ganji to get a value for t
t = 3.182;
tPv = (tvoltdev*3.182)./sqrt(4); cPv = (cvoltdev*3.182)./sqrt(4); % Random uncertainty for mean voltage
% rssq.m only works for vectors that match in size, so we have to manually write:
twv = sqrt(tPv.^2 + Bv^2); cwv = sqrt(cPv.^2 + Bv^2); % Uncertainty in voltage

Bwind1 = (anemometer_readings*0.008) + (0.001*3); % Multimeter accuracy of ± (0.8% + 3)
                                                  % And resolution error of 0.001
Bwind1 = 20.25 * Bwind1; % Convert voltage error to wind speed; offset doesn't affect error
% because sensitivity coefficient is the partial derivative of the output WRT the input
Bwind2 = 0.3 + wind_speeds*0.03; % Accuracy of ± (0.3+0.03V) m/s on the anemometer, V is current speed
Bwind = sqrt(Bwind1.^2 + Bwind2.^2); % Total systematic uncertainty
% Use a similar process as above for random uncertainty on mean, t value stays the same
for ii = 1:4
    speeddev(ii) = std(all_wind_speeds(ii,:)); % Std dev of speeds
end
Pwind = (t*speeddev)./sqrt(4);
wwind = sqrt(Pwind.^2 + Bwind.^2);

% Power in from wind
P_in_t = (1 / 2) * rho * (wind_speeds .^ 3) * areaT; % Watts
P_in_c = (1 / 2) * rho * (wind_speeds .^ 3) * areaC;   % Watts
% Electrical power out
P_out_t = (Vout_t .^ 2) / R_load; % Watts
P_out_c = (Vout_c .^ 2) / R_load; % Watts

% Efficiency (dimensionless)
```

```
eta_t = P_out_t ./ P_in_t; % 1x4 double

weta_t = eta_t .* sqrt( (2*twv ./ Vout_t).^2 + (wr / R_load)^2 + (wrho / rho)^2 + ...
                        (wareaT / areaT)^2 + (3*wwind ./ wind_speeds).^2 );
eta_c = P_out_c ./ P_in_c;
weta_c = eta_c .* sqrt( (2*cwv ./ Vout_c).^2 + (wr / R_load)^2 + (wrho / rho)^2 + ...
                        (wareaC / areaC)^2 + (3*wwind ./ wind_speeds).^2 );

eta_t_percent = 100 * eta_t;
weta_t_percent = 100 * weta_t;
eta_c_percent = 100 * eta_c;
weta_c_percent = 100 * weta_c;
% fprintf('\nToroidal efficiency values (%%):\n')
% disp(eta_t_percent)
% fprintf('Curved efficiency values (%%):\n')
% disp(eta_c_percent)
%% Efficiency vs Wind Speed plot
figure;  hold on; grid on; box on
col_C = [0.00, 0.45, 0.70]; % Blue for curved
col_T = [0.00, 0.00, 0.00]; % Black for toroidal
col_w = [0.65, 0.65, 0.65]; % Gray for wind speed

wbar = errorbar(wind_speeds,eta_t_percent,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None', ...
    'DisplayName','Wind Speed (mean \pm \omega_{\mathcal{V}})');
errorbar(wind_speeds,eta_c_percent,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None','HandleVisibility','Off')
tbar = errorbar(wind_speeds,eta_t_percent,weta_t_percent,'s','Color',col_T, ...
    'MarkerFaceColor',col_T,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Toroidal (mean \pm \omega_\eta)');
cbar = errorbar(wind_speeds,eta_c_percent,weta_c_percent,'o','Color',col_C, ...
    'MarkerFaceColor',col_C,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Curved (mean \pm \omega_\eta)');
axes = gca; axes.FontSize = 12; axes.FontWeight = 'bold';
xlabel('Wind Speed (m/s)','Fontweight','bold','FontSize',16);
ylabel('Efficiency (%)','fontweight','bold','Fontsize',16);
legend([tbar,cbar,wbar],'Location','Northeast','FontSize',10,'FontWeight','Bold');
title('Efficiency \eta vs Wind Speed','FontSize',18);
%% Tip-speed ratio vs Wind Speed
% lambda = (radius * omega) / V Formula for tip-speed ratio (tsr)
% omega = 2 * pi * (RPM / 60)
tsr_t = (radT * 2 * pi * rpm_t / 60) ./ wind_speeds;
tsr_c = (radC * 2 * pi * rpm_c / 60) ./ wind_speeds;

% Uncertainty analysis for RPM
Brpm = 30; % Rotational resolution error
allrpm_c = [c425mean_rpm ; c599mean_rpm ; c790mean_rpm; c968mean_rpm];
allrpm_t = [t425mean_rpm ; t599mean_rpm ; t790mean_rpm; t968mean_rpm];
% ^ Each row is a wind speed, each column is the trial
for ii = 1:4
    crpmdev(ii) = std(allrpm_c(ii,:)); % Std dev of rpm. Each column is a speed
    trpmdev(ii) = std(allrpm_t(ii,:));
end
cPrpm = (t*crpmdev)./sqrt(4); tPrpm = (t*trpmdev)./sqrt(4); % Random uncertainty in mean RPM. t is still the same
cwrpm = sqrt(cPrpm.^2 + Brpm.^2); twrpm = sqrt(tPrpm.^2 + Brpm.^2); % Total uncertainties

% Uncertainties in the tip speed ratios
twtsr = tsr_t .* sqrt( (wwind./wind_speeds).^2 + (twrpm./rpm_t).^2 + (wradT./radT)^2);
cwtsr = tsr_c .* sqrt( (wwind./wind_speeds).^2 + (cwrpm./rpm_c).^2 + (wradC./radC)^2);

% fprintf('Toroidal tip-speed ratio values:\n'); disp(tsr_t)
% fprintf('Curved tip-speed ratio values:\n'); disp(tsr_c)
figure; hold on; grid on; box on
wbar = errorbar(wind_speeds,tsr_t,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None', ...
    'DisplayName','Wind Speed (mean \pm \omega_{\mathcal{V}})');
errorbar(wind_speeds,tsr_c,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None','HandleVisibility','Off')
tbar = errorbar(wind_speeds,tsr_t,twtsr,'s','Color',col_T, ...
    'MarkerFaceColor',col_T,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Toroidal (mean \pm \omega_\lambda)');
cbar = errorbar(wind_speeds,tsr_c,cwtsr,'o','Color',col_C, ...
    'MarkerFaceColor',col_C,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Curved (mean \pm \omega_\lambda)');
axes = gca; axes.FontSize = 12; axes.FontWeight = 'bold';

xlabel('Wind Speed (m/s)','Fontweight','bold','FontSize',16);
ylabel('Tip-Speed Ratio','Fontweight','bold','FontSize',16);
title('Tip-Speed Ratio \lambda vs Wind Speed','FontSize',18);
ylim([0.8 3.4])
legend([tbar,cbar,wbar],'Location','South','FontSize',10,'FontWeight','Bold');
```

2. ErrorBetterBladesv2.m, used for analyzing sound data

```
%% Processing sound data and computing uncertainty analysis script
% ErrorBetterBlades_V2, written 19 April 2026 by Eileen Phan and Aaron Tran
% The purpose of this script is to process the data through time domain
% subtraction since sound pressure is additive.
% Nomenclature:
% w (at the start of a variable) = Total uncertainty
% B (at the start of a variable) = Systematic uncertainty
% P (at the start of a variable) = Random uncertainty
% n (natural number at the end of a variable) = Used if a variable has more
%            than one type of a type of uncertainty e.g. Btemp1 and Btemp2
% C (at the end of a variable) = Belongs to curved blade data
% T (at the end of a variable) = Belongs to toroidal blade data
%% Sound Pressure Level Processing
% METHOD: Time-domain background subtraction to get isolated blade SPL
% sig_isolated = sig_blade(1:N) - sig_background(1:N)
% P_rms = rms(sig_isolated)
% L_p = 20*log10(P_rms / P_ref)
% This removes the motor's physical pressure contribution
% giving true isolated blade noise. The mic scale factor cancels since both
% signals are recorded with the same microphone, so no calibration is
% needed for comparative study purposes.
Pref = 20*10^-6; % Reference pressure (Pa)
speeds = [4 6 8 9.5]; % Different speeds tested
% For the 4x4 matrices initialized below: each row = trial. Each column = speed
soundC = cell(4,4); backgroundC = cell(4,1);
soundT = cell(4,4); backgroundT = cell(4,1);
% Import the audio data for the blades
for i = 1:4 % 4 trials per speed
    trial = num2str(i);
    for ii = 1:length(speeds) % Import data and put in appropriate matrices
        speed = num2str(speeds(ii)); % temp means temporary in this context
        temp = load(['curved_' trial '_' speed 'ms'],'audio_data');
        soundC{i,ii} = double(temp.audio_data(:));
        if i == 4 && ii == 4 % This trial was missing
            soundT{i,ii} = [];
        else
            temp = load(['toroid_' trial '_' speed 'ms'],'audio_data');
            soundT{i,ii} = double(temp.audio_data(:));
        end
    end
end
% This loop imports the background audio data, which is slightly inconsistent
for ii = 1:length(speeds)
    speed = num2str(speeds(ii));
    if ii == 2 % 6 m/s data had 4 background trials for curved and 3 for toroidal
        tempC = cell(1,4); tempT = cell(1,3); % Temporary variables
        for i = 1:4 % Take average across 4 trials
            trial = num2str(i);
            temp = load(['background_curved_' trial '_' speed 'ms'],'audio_data');
            tempC{i} = double(temp.audio_data(:)); % Temporary variable
        end
        backgroundC{ii} = mean(cell2mat(tempC),2); % Average across columns
        for i = 1:3 % Take average across 3 trials
            trial = num2str(i);
            temp = load(['background_toroidal_' trial '_' speed 'ms'],'audio_data');
            tempT{i} = double(temp.audio_data(:)); % Temporary variable
        end
        backgroundT{ii} = mean(cell2mat(tempT),2); % Average across columns
    else % Other data only had 1 background trial that lasted 4 minutes
        temp = load(['background_curved_1_' speed 'ms'],'audio_data');
        backgroundC{ii} = double(temp.audio_data(:));
        temp = load(['background_toroidal_1_' speed 'ms'],'audio_data');
        backgroundT{ii} = double(temp.audio_data(:));
    end % In the future it would be better to make background a row vector
end % so that it matches the notion of speed differing per column
% Subtract background in time domain, then compute RMS and SPL
splC = nan(4,4); % 4 trials x 4 speeds
splT = nan(4,4);
```

```
for ii = 1:length(speeds)
    bg_C = backgroundC{ii};
    bg_T = backgroundT{ii};
    for i = 1:4
        % Curved blade. Extract the blade+background recording for trial i
        % at speed ii
        sig = soundC{i,ii};
        if ~isempty(sig) % skip over missing files
            N = min(length(sig), length(bg_C)); % trim the background so that same lengths
            % p_measured(t) = p_blade(t) + p_motor(t)
            % p_blade(t) = p_measured(t) - p_motor(t)
            sig_iso = sig(1:N) - bg_C(1:N); % time-domain subtraction of isolated signal
            splC(i,ii) = 20 * log10(rms(sig_iso) / Pref); % arbitrary units cancel
        end
        % Toroidal
        sig = soundT{i,ii};
        if ~isempty(sig)
            N = min(length(sig), length(bg_T));
            sig_iso = sig(1:N) - bg_T(1:N); % time-domain subtraction
            splT(i,ii) = 20 * log10(rms(sig_iso) / Pref);
        end
    end
end
%% Sound Pressure Level Error (Random uncertainty)
% Random uncertainty in a single measurement is t*s
% where s is the standard deviation and t is the t-value
ssplC = std(splC)'; ssplT = std(splT,'omitnan')'; % Gets the std deviation across each trial
% v = degrees of freedom (n-1)
% a = 1 - confidence level, so 0.05 for a confidence of 95%. a/2 = 0.025
% Use v and a/2 for Table 6.6 on Page 158 in Introduction to Engineering
% Experimentation by Wheeler and Ganji to get a value for t
% 4 trials, v = 3, t = 3.182
% 3 trials, v = 2, t = 4.303
PsplC = ssplC*3.182;
PsplT = ssplT*3.182; PsplT(4) = ssplT(4)*4.303; % Since toroid 9.5 m/s had 3 trials
% Total uncertainty = random only
% (mic scale cancels in time-domain subtraction, no systematic mic term)
```

3. SPL_vs_WindSpeed_RPM.m, used to create Figures 8 and the left side of Figure 9.

```
%% SPL vs Wind Speed - Ever Better Blades
% SPL_vs_WindSpeed_RPM.m
% Written April 2026 by Eileen Phan
%
% Before using this script run ErrorBetterBlates_V2
% Plots raw blade+background SPL vs wind speed for curved (blue) and
% toroidal (black) blades with random uncertainty error bars.
% splC, splT : 4 trials, 4 speeds (dB)
% PsplC, PsplT: 4x1, random uncertainty in SPL per speed
close all
load('allerrors.mat', 'wwind')
load('RelevantMatrices.mat', 'wind_speeds')
%% Mean across the trials
mean_C = mean(splC, 1, 'omitnan');   % 1 x 4
mean_T = mean(splT, 1, 'omitnan');
%% Plot
col_C = [0.00, 0.45, 0.70];% blue for curved
col_T = [0.00, 0.00, 0.00];% black for toroidal
col_w = [0.65, 0.65, 0.65]; % Gray for wind speed

figure('Name', 'Blade SPL vs Wind Speed', ...
       'NumberTitle', 'off');
hold on; grid on; box on;

% Individual trial scatter points
% Commented out because although it's good to do, it wasn't very visible
% for i = 1:size(splC, 1)
%     valid_C = ~isnan(splC(i,:));
%     scatter(wind_speeds(valid_C), splC(i, valid_C), 35, col_C, 'o', ...
%             'MarkerFaceColor', col_C, 'MarkerFaceAlpha', 0.20, ...
%             'MarkerEdgeColor', 'none', 'HandleVisibility', 'off');
% end
% for i = 1:size(splT, 1)
%     valid_T = ~isnan(splT(i,:));
%     scatter(wind_speeds(valid_T), splT(i, valid_T), 35, col_T, 's', ...
%             'MarkerFaceColor', col_T, 'MarkerFaceAlpha', 0.20, ...
```

```
%             'MarkerEdgeColor', 'none', 'HandleVisibility', 'off');
% end
% Mean + random uncertainty error bars
wbar = errorbar(wind_speeds,mean_T,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None', ...
    'DisplayName','Wind Speed (mean \pm \omega_{\mathcal{V}})');
errorbar(wind_speeds,mean_C,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None','HandleVisibility','Off')
tbar = errorbar(wind_speeds, mean_T, PsplT', 's', ...
         'Color', col_T, 'MarkerFaceColor', col_T, ...
         'MarkerSize', 7, 'LineWidth', 1, 'CapSize', 8, ...
         'DisplayName', 'Toroidal (mean \pm P_{SPL})');
cbar = errorbar(wind_speeds, mean_C, PsplC', 'o', ...
         'Color', col_C, 'MarkerFaceColor', col_C, ...
         'MarkerSize', 7, 'LineWidth', 1, 'CapSize', 8, ...
         'DisplayName', 'Curved (mean \pm P_{SPL})');

axes = gca; axes.FontSize = 12; axes.FontWeight = 'Bold';
xlabel('Wind Speed (m/s)', 'FontSize', 21,'FontWeight','Bold');
ylabel('Blade SPL (dB)', 'FontSize', 21,'FontWeight','Bold');
legend([tbar,cbar,wbar],'Location','southeast','FontSize',12,'FontWeight','Bold');
title('Blade SPL vs Wind Speed','FontSize', 20, 'FontWeight', 'bold');
xlim([3.5, 11.5]);
%% Print summary of results
% fprintf('Blade SPL (dB) \n');
% fprintf('  Speed   |   Curved mean ± P_spl  |  Toroidal mean ± P_spl \n');
% for si = 1:length(wind_speeds)
%     fprintf('  %4g m/s |   %6.2f ± %5.2f dB    |   %6.2f ± %5.2f dB    \n', ...
%             wind_speeds(si), mean_C(si), PsplC(si), mean_T(si), PsplT(si));
% end

%% RPM vs SPL Normalization Plot
% This script plots isolated blade SPL against rotor RPM for both blade
% geometries, normalizing out the wind speed dependence so that both
% designs are compared at their equivalent rotational operating points.

% SPL values from SPL_vs_WindSpeed.m output (mean across trials)
% RPM values from testdatawitherror.m (hall sensor counts * 15)
clear;
load('allerrors','cwrpm','twrpm')
% Index dardcoded data - 4, 6, 8, 9.5 m/s
% from hall sensor data
rpm_c = [427.0,577.1,807.6,914.7]; % Curved RPM
rpm_t = [651.8,867.8, 1183.6,1331.8]; % Toroidal RPM
% from audio files and spl_vs_windspeed
spl_c = [60.48,65.60,76.19,79.77]; % Curved mean SPL (dB)
spl_t = [69.11,79.92,83.82,85.37]; % Toroidal mean SPL (dB)
% random uncertainties from ErrorBetterBlades_V2.m
Pspl_c = [0.65,0.25,0.43,0.40]; % Curved
Pspl_t = [2.56,0.56,0.22,0.34]; % Toroidal

% add the log scale,
% Curve fits (log-linear: SPL = a*log(RPM) + b)
% Fit in log b/c noise scales w/ RPM
% spl = m*log(RPM)+y
p_c = polyfit(log(rpm_c), spl_c, 1); % first order passes through log(rpm_c)
p_t = polyfit(log(rpm_t), spl_t, 1);
% Overlapping RPM range for interpolation
% Curved has 427-915 rpm, toroid 652-915 rpm, interlaps between 652-915 rpm
rpm_min = max(min(rpm_c), min(rpm_t));% 651.8 RPM
rpm_max = min(max(rpm_c), max(rpm_t));% 914.7 RPM
% Make fits across full range and overlap range
rpm_overlap = linspace(rpm_min, rpm_max, 200); % create 200 rpm values within min & max
rpm_full_c = linspace(min(rpm_c)*0.95, max(rpm_c)*1.05, 200);
rpm_full_t = linspace(min(rpm_t)*0.95, max(rpm_t)*1.05, 200);
spl_fit_c = polyval(p_c, log(rpm_full_c)); % take fitted line and eval at RPM values
spl_fit_t = polyval(p_t, log(rpm_full_t));
spl_overlap_c = polyval(p_c, log(rpm_overlap));
spl_overlap_t = polyval(p_t, log(rpm_overlap));
% SPL difference at matched RPM within overlap region
spl_diff = spl_overlap_t - spl_overlap_c;
% R-squared for each fit
ss_res_c = sum((spl_c - polyval(p_c, log(rpm_c))).^2);
ss_tot_c = sum((spl_c - mean(spl_c)).^2);
ss_res_t = sum((spl_t - polyval(p_t, log(rpm_t))).^2);
ss_tot_t = sum((spl_t - mean(spl_t)).^2);
fprintf('\nCurved R²:   %.4f\n', 1 - ss_res_c/ss_tot_c);
fprintf('Toroidal R²: %.4f\n', 1 - ss_res_t/ss_tot_t);

% Plot - enhanced with Claude
col_C = [0.00, 0.45, 0.70]; % blue
col_T = [0.00, 0.00, 0.00]; % black
col_shaded = [0.85, 0.85, 0.85]; % light grey for overlap region
figure('Name', 'RPM vs SPL', 'NumberTitle', 'off', 'Position', [100 100 750 520]);
```

```
hold on; grid on; box on;
% Shade the overlapping RPM region
yFill = [50 95];
fill([rpm_min rpm_max rpm_max rpm_min], ...
     [yFill(1) yFill(1) yFill(2) yFill(2)], ...
     col_shaded, 'EdgeColor', 'none', 'FaceAlpha', 0.4, ...
    'DisplayName', 'Overlapping RPM range');
% barrpm = errorbar(rpm_c,spl_c,cwrpm,'Horizontal','Color',col_C,'LineWidth',1.2, ...
%     )
% Fitted curves (dashed, full extrapolated range)
plotc = plot(rpm_full_c, spl_fit_c, '--', 'Color', col_C, 'LineWidth', 1.2, ...
'DisplayName', 'Curved fit');
plott = plot(rpm_full_t, spl_fit_t, '--', 'Color', col_T, 'LineWidth', 1.2, ...
 'DisplayName', 'Toroidal fit');
% Fitted curves within overlap (solid, thicker)
fitc = plot(rpm_overlap, spl_overlap_c, '-', 'Color', col_C, 'LineWidth', 2.2, ...
 'HandleVisibility', 'off');
fitt = plot(rpm_overlap, spl_overlap_t, '-', 'Color', col_T, 'LineWidth', 2.2, ...
 'HandleVisibility', 'off');
% Data points with error bars
barc = errorbar(rpm_c,spl_c,Pspl_c,Pspl_c,cwrpm,cwrpm, 'o', ...
 'Color', col_C, 'MarkerFaceColor', col_C, ...
    'MarkerSize', 7, 'LineWidth', 1.2, ...
         'DisplayName', sprintf(['Curved (mean \\pm P_{SPL}) \n ' ...
         '          (mean \\pm \\omega_{RPM})']));
bart = errorbar(rpm_t, spl_t, Pspl_t,Pspl_t,twrpm,twrpm, 's', ...
'Color', col_T, 'MarkerFaceColor', col_T, ...
    'MarkerSize', 7, 'LineWidth', 1.2, ...
         'DisplayName', sprintf(['Toroidal (mean \\pm P_{SPL}) \n ' ...
         '           (mean \\pm \\omega_{RPM})']));
% Annotate wind speeds next to data points
% Each label is manually offset to avoid overlapping data points and each other
wind_speeds = {'4.25 m/s', '5.99 m/s', '7.90 m/s', '9.68 m/s'};
%           4 m/s    6 m/s    8 m/s    9.5 m/s
x_off_c = [  40,   -15,   -8,    15];  % negative = left of point
y_off_c = [    2,     2,    1.7,     2];  % positive = above point
x_off_t = [   -130,    -90,    -100,    -100];
y_off_t = [   1.9,   2,     1.8,     1.8];
halign_c = {'right','right','right','left'};  % 9.5 m/s curved flips right to avoid toroid
for k = 1:4
    text(rpm_c(k) + x_off_c(k), spl_c(k) + y_off_c(k), wind_speeds{k}, ...
         'FontSize', 8, 'Color', col_C, 'FontWeight', 'bold', ...
         'HorizontalAlignment', halign_c{k});
    text(rpm_t(k) + x_off_t(k), spl_t(k) + y_off_t(k), wind_speeds{k}, ...
         'FontSize', 8, 'Color', col_T, 'FontWeight', 'bold', ...
         'HorizontalAlignment', 'left');
end
xlim([350 1450]);
ylim([55 92]);
axes = gca; axes.FontSize = 12; axes.FontWeight = 'Bold';
xlabel('Rotor Speed (RPM)','FontWeight','Bold','FontSize',21);
ylabel('Blade SPL (dB)','FontWeight','Bold','FontSize',21);
legend('Location', 'southeast', 'FontSize', 11);
title('Blade SPL vs Rotor Speed','FontSize',20);
set(gcf,'Position',[573 337.6667 560 420])
```

4. SPL_vs_WindSpeed_Energy.m, used to normalize sound data and create Figures 10 and 11.

```
clear; close all;
%% SPL vs Wind Speed - Ever Better Blades
% SPL_vs_WindSpeed_Jose.m
% Written April 2026 by Jose Gutierrez
%
% Plots SPl normalized by energy per volumetric flow rate vs wind speed for
% curved (blue) and toroidal (black) blades with random uncertainty error bars.
% splC, splT : 4 trials, 4 speeds (dB)
% PsplC, PsplT: 4x1, random uncertainty in SPL per speed

%% Loading SPL and Phase 1 data
spl_data = load('spl_data.mat');
splC = spl_data.splC;
splT = spl_data.splT;
PsplC = spl_data.PsplC;
PsplT = spl_data.PsplT;
phase_1_testing_data = load('phase_1_testing_data.mat');
Vout_t = phase_1_testing_data.Vout_t_save;
Vout_c = phase_1_testing_data.Vout_c_save;
wind_speeds = phase_1_testing_data.wind_speeds;
load('allerrors')
%% Mean across the trials
mean_C = mean(splC, 1, 'omitnan'); % 1 x 4
mean_T = mean(splT, 1, 'omitnan');
% Plot
```

```
col_C = [0.00, 0.45, 0.70]; % blue for curved
col_T = [0.00, 0.00, 0.00]; % black for toroidal
col_w = [0.65, 0.65, 0.65]; % Gray for wind speed
%% Print summary of results

% Force all 4-point datasets to column vectors
mean_C = mean_C(:);
mean_T = mean_T(:);

% fprintf('Blade SPL (dB) \n');
% fprintf('  Speed   |   Curved mean ± P_spl  |  Toroidal mean ± P_spl \n');
% for si = 1:length(wind_speeds)
%     fprintf('  %4g m/s |   %6.2f ± %5.2f dB    |   %6.2f ± %5.2f dB    \n', ...
%             wind_speeds(si), mean_C(si), PsplC(si), mean_T(si), PsplT(si));
% end

%% Calculating and Plotting Normalized Sound Metric
%
p_ref = 20e-6; % reference pressure used to solve for p_rms
p_rms_curved = p_ref * 10 .^ (mean_C / 20);
p_rms_toroidal = p_ref * 10 .^ (mean_T / 20);
wrms_t = (PsplT .* (log(10) * p_ref * 10.^(mean_T./20))) / 20; % Uncertainty
wrms_c = ( PsplC .* (log(10) * p_ref * 10.^(mean_C./20)) ) / 20; % using partial deriv
%
% Calculating volumetric flow rate of both designs
radius_curved = 177.01 / 1000; % m
radius_toroidal = 177.11 / 1000; % m
A_curved = pi * radius_curved ^ 2; % m^2
A_toroidal = pi * radius_toroidal ^ 2; % m^2
vol_flow_rate_curved = wind_speeds * A_curved; % (m^3)/s
vol_flow_rate_toroidal = wind_speeds * A_toroidal; % (m^3)/s
wvolflow_c = vol_flow_rate_curved' .* sqrt( (wwind./wind_speeds').^2 ...
    + (wareaC/A_curved)^2 ); % Uncertainty in
wvolflow_t = vol_flow_rate_toroidal' .* sqrt( (wwind./wind_speeds').^2 ...
    + (wareaT/A_toroidal)^2 ); % vol flow rate

% Calculating power output of both designs
resistance = 1000; % ohms
power_curved = (Vout_c .^ 2) / resistance; % watts
power_toroidal = (Vout_t .^ 2) / resistance; % watts

% Calculating normalized sound metric (unitless)
normalized_sound_curved = (p_rms_curved .* vol_flow_rate_curved) ./ power_curved;
normalized_sound_toroidal = (p_rms_toroidal .* vol_flow_rate_toroidal) ./ power_toroidal;
% Uncertainty in normalized sound
wnormalizedsound_c = normalized_sound_curved' .* sqrt((wrms_c'./p_rms_curved').^2 + ...
    (wvolflow_c./vol_flow_rate_curved').^2 + (2*cwv./Vout_c').^2 + (wr/resistance)^2);
wnormalizedsound_t = normalized_sound_toroidal' .* sqrt((wrms_t'./p_rms_toroidal').^2 + ...
    (wvolflow_t./vol_flow_rate_toroidal').^2 + (2*twv./Vout_t').^2 + (wr/resistance)^2);
%% Plot normalized sound vs wind speed
% close all
figure; hold on; grid on; box on
wbar = errorbar(wind_speeds,normalized_sound_toroidal,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None', ...
    'DisplayName','Wind Speed (mean \pm \omega_{\mathcal{V}})');
errorbar(wind_speeds,normalized_sound_curved,wwind,'horizontal','o','Color',col_w, ...
    'LineWidth',1,'CapSize',8,'Marker','None','HandleVisibility','Off')
tbar = errorbar(wind_speeds,normalized_sound_toroidal,wnormalizedsound_t,'s','Color',col_T, ...
    'MarkerFaceColor',col_T,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Toroidal (mean \pm \omega_{norm})');
cbar = errorbar(wind_speeds,normalized_sound_curved,wnormalizedsound_c,'o','Color',col_C, ...
    'MarkerFaceColor',col_C,'MarkerSize',7,'LineWidth',1,'CapSize',8, ...
    'DisplayName','Curved (mean \pm \omega_{norm})');
axes = gca; axes.FontSize = 12; axes.FontWeight = 'Bold';
xlabel('Wind Speed (m/s)','FontWeight','Bold','Fontsize',21);
ylabel('Normalized Sound Metric','FontWeight','Bold','Fontsize',21);
legend([tbar,cbar,wbar], 'Location', 'northwest','FontSize',12,'FontWeight','Bold');
title('Normalized Sound vs Wind Speed','FontSize',20);

%% Normalized Sound vs Reynolds Number (Diameter as characteristic length)
% Reynolds numbers recomputed using rotor diameter as characteristic length
% Reference turbine: D_ref = 0.94 m, Rochester wind speeds 3.13-4.47 m/s
% Lower bound extended 20% downward, upper bound extended 5% upward
% to account for dynamic similarity tolerance

D_ref = 0.94;                                % m, reference rotor diameter
D_t = 2 * 177.11 / 1000;                     % m, toroidal diameter
D_c = 2 * 177.01 / 1000;                     % m, curved diameter
mu_true = 0.00001839;                        % Pa*s, lab dynamic viscosity
rho_true = 1.192;                            % kg/m^3, lab air density
rho_ref = 1.225;                             % kg/m^3, reference density
mu_ref = 1.825e-5;                           % Pa*s, reference dynamic viscosity

% Reference Re range using reference diameter and Rochester wind speeds
```

```
Re_ref_low  = (rho_ref * 3.13 * D_ref) / mu_ref;
Re_ref_high = (rho_ref * 4.47 * D_ref) / mu_ref;

% Tolerance bounds
Re_lower_bound = Re_ref_low  * 0.80; % 20% decrease on lower
Re_upper_bound = Re_ref_high * 1.05; % 5% increase on upper

% Tested Re values
Re_t = (rho_true .* wind_speeds * D_t) / mu_true;
Re_c = (rho_true .* wind_speeds * D_c) / mu_true;

idx = [3, 4]; % 7.90 and 9.68 m/s

% Force everything to row vectors of length 2
ns_t = normalized_sound_toroidal(:)'; ns_t = ns_t(idx);
ns_c = normalized_sound_curved(:)';   ns_c = ns_c(idx);
wns_t = wnormalizedsound_t(:)';       wns_t = wns_t(idx);
wns_c = wnormalizedsound_c(:)';       wns_c = wns_c(idx);
Ret = Re_t(:)'; Ret = Ret(idx);
Rec = Re_c(:)'; Rec = Rec(idx);

% Uncertainty in the Re calculations
newwind = wwind(idx); % Take the 7.90 and 9.68 wind speed uncertainties
newwind_speeds = wind_speeds(idx)'; % Likewise for wind speeds
wRet = Ret.*sqrt((wrho/rho_true)^2 + (newwind./newwind_speeds).^2 + ...
    (wradT/D_t)^2); % R = 2D so diameter uncertainty = radius uncertainty
wRec = Rec.*sqrt((wrho/rho_true)^2 + (newwind./newwind_speeds).^2 + ...
    (wradC/D_c)^2);
% ^ Dynamic viscosity has no error since we're taking it as an accepted value

% Compute y limits explicitly from data
y_low  = min([ns_t - wns_t, ns_c - wns_c]) * 0.85;
y_high = max([ns_t + wns_t, ns_c + wns_c]) * 1.15;

% Print Re values for reference
fprintf('\nReference Re range (D = %.2f m): %.0f - %.0f\n', D_ref, Re_ref_low, Re_ref_high);
fprintf('Extended Re bounds (-20%% / +5%%): %.0f - %.0f\n', Re_lower_bound, Re_upper_bound);
fprintf('Toroidal Re at tested speeds: %.0f, %.0f, %.0f, %.0f\n', Re_t);
fprintf('Curved Re at tested speeds:   %.0f, %.0f, %.0f, %.0f\n\n', Re_c);

figure; hold on; grid on; box on;

% Original reference Re range (darker)
fill([Re_ref_low Re_ref_high Re_ref_high Re_ref_low], ...
     [y_low y_low y_high y_high], ...
    [0.65 0.65 0.65], 'EdgeColor', 'none', 'FaceAlpha', 0.5, ...
    'DisplayName', sprintf('Reference Re range (%.0f-%.0f)', Re_ref_low, Re_ref_high));

% Lower bound extension (-20%)
fill([Re_lower_bound Re_ref_low Re_ref_low Re_lower_bound], ...
     [y_low y_low y_high y_high], ...
    [0.85 0.85 0.85], 'EdgeColor', 'none', 'FaceAlpha', 0.5, ...
    'DisplayName', 'Extended Re range (-20% lower bound)');

errorbar(Ret, ns_t, wns_t, wns_t, wRet, wRet, ...
    's', 'Color', col_T, 'MarkerFaceColor', col_T, 'MarkerSize', 10, ...
    'LineWidth', 1.2, 'CapSize', 8, ...
    'DisplayName', sprintf(['Toroidal (mean \\pm \\omega_{norm}) \n ' ...
    '              (nominal \\pm \\omega_{Re})']));
errorbar(Rec, ns_c, wns_c, wns_c, wRec, wRec, ...
    'o', 'Color', col_C, 'MarkerFaceColor', col_C, 'MarkerSize', 10, ...
    'LineWidth', 1.2, 'CapSize', 8, ...
    'DisplayName', sprintf(['Curved (mean \\pm \\omega_{norm}) \n ' ...
    '            (nominal \\pm \\omega_{Re})']));

ax = gca; ax.FontSize = 12; ax.FontWeight = 'Bold';
xlabel('Reynolds Number', 'FontWeight', 'Bold', 'FontSize', 21);
ylabel('Normalized Sound Metric', 'FontWeight', 'Bold', 'FontSize', 21);
legend('Location', 'NorthWest', 'FontSize', 11, 'FontWeight', 'Bold');
title('Normalized Sound vs Reynolds Number', 'FontSize', 18);

% Set axis limits with some padding around data and shaded regions
all_x = [Re_lower_bound, Re_upper_bound, Ret, Rec];
xlim([min(all_x)*0.95, 2.9E5]);
ylim([10 38])
```

Appendix F: Miscellaneous Information Related to Measurements

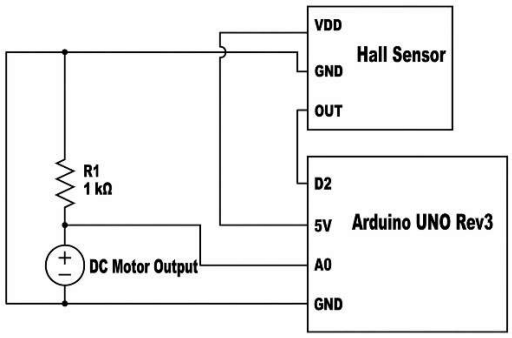


Figure 17: Circuit diagram used to measure both RPM and voltage output of turbine.



Figure 18: The Adafruit anemometer used in this project to measure wind speed.

1. Python code for calculating moment of inertia in the blade designs

```
from stl import mesh
import numpy as np

# Load stl
InputMesh = mesh.Mesh.from_file(r'C:\programs\nxx\Wind turbine\CurvedBladeGeometry.stl')

# Functions
def tetra_signed_volume(a, b, c):
    return np.dot(a, np.cross(b, c)) / 6.0


def tetra_centroid(a, b, c):
    # Tetrahedron with vertices [0, a, b, c]
    return (a + b + c) / 4.0


def tetra_inertia_about_origin(a, b, c, density):
    """
    Inertia tensor of tetrahedron with vertices [0, a, b, c]
    about the origin, for uniform density.
    """
    V = tetra_signed_volume(a, b, c)
    m = density * V

    x1, y1, z1 = a
    x2, y2, z2 = b
    x3, y3, z3 = c

    def f2(u1, u2, u3):
        return u1*u1 + u2*u2 + u3*u3 + u1*u2 + u2*u3 + u3*u1

    def g(u1, u2, u3, v1, v2, v3):
        return (
            2*u1*v1 + 2*u2*v2 + 2*u3*v3
            + u1*v2 + u2*v1
            + u1*v3 + u3*v1
            + u2*v3 + u3*v2
```

```
        )

    # Diagonal terms
    Ixx = m / 10.0 * (f2(y1, y2, y3) + f2(z1, z2, z3))
    Iyy = m / 10.0 * (f2(x1, x2, x3) + f2(z1, z2, z3))
    Izz = m / 10.0 * (f2(x1, x2, x3) + f2(y1, y2, y3))

    # Products of inertia
    Ixy = -m / 20.0 * g(x1, x2, x3, y1, y2, y3)
    Ixz = -m / 20.0 * g(x1, x2, x3, z1, z2, z3)
    Iyz = -m / 20.0 * g(y1, y2, y3, z1, z2, z3)

    return np.array([
        [Ixx, Ixy, Ixz],
        [Ixy, Iyy, Iyz],
        [Ixz, Iyz, Izz]
    ]), V


def mesh_mass_properties(InputMesh, density):
    total_vol = 0.0
    first_moment = np.zeros(3)
    I_origin = np.zeros((3, 3))

    for tri in InputMesh.vectors:
        a, b, c = tri[0], tri[1], tri[2]

        V = tetra_signed_volume(a, b, c)
        centroid_tet = tetra_centroid(a, b, c)
        I_tet, _ = tetra_inertia_about_origin(a, b, c, density=density)

        total_vol += V
        first_moment += V * centroid_tet
        I_origin += I_tet

    # Signed volume fix
    if total_vol < 0:
        total_vol = -total_vol
        first_moment = -first_moment
        I_origin = -I_origin

    centroid = first_moment / total_vol
    mass = density * total_vol

    # Parallel axis theorem
    r = centroid
    r2 = np.dot(r, r)
    I_centroid = I_origin - mass * (r2 * np.eye(3) - np.outer(r, r))

    return total_vol, mass, centroid, I_origin, I_centroid


# run - the one we need is Izz (entry (3,3))
density = 0.00000114703591519  # user input
volume, mass, centroid, I_origin, I_centroid = mesh_mass_properties(InputMesh, density=density)

print(f"Volume: {volume}")
print(f"Mass: {mass}")
print(f"Center of mass: {centroid}")

print("\nInertia tensor about center of mass:")
print(I_centroid)

# Principal moments of inertia
eigvals, eigvecs = np.linalg.eigh(I_centroid)
print("\nPrincipal moments of inertia:")
print(eigvals)

print("\nPrincipal axes (columns):")
print(eigvecs)
```

2. Arduino code used for data collection

```
// Data collection for efficiency and TSR comparison

void hall_sensor_interrupt(); // function prototype for the hall sensor interrupt


volatile unsigned int hall_sensor_counts = 0; // stores the number of hall sensor triggers

volatile uint8_t timer_cycles = 0; // stores the current sample index from 0 to 59

volatile unsigned int hall_sensor_array[61]; // stores the number of hall sensor triggers during each 1-second interval of the trial
```

```
volatile unsigned int generator_array[61]; // stores the ADC reading of the generator voltage once per second

void setup()
{
  Serial.begin(9600); // setting the baud rate to 9600 bits per second
  pinMode(2, INPUT_PULLUP); // setting external interrupt pin (pin number 2) to be a digital input and setting it's default value to HIGH
  pinMode(4, INPUT_PULLUP); // setting pull up resistor pin for synching Arduinos (pin number 4) to be a digital input
  cli(); // disabling interrupts so that they don't trigger during the configuration of them

  attachInterrupt(digitalPinToInterrupt(2), hall_sensor_interrupt, FALLING); // creating the hall sensor interrupt and setting it to trigger on a falling edge

  // Below is the configuration for the timer interrupt
  TCCR1A = 0; // resetting control register A
  TCCR1B = 0; // resetting control register B
  TCCR1B |= (1 << WGM12); // set CTC mode
  OCR1A = 15624; // setting compare match value to 15624 so timer triggers every second
  TCCR1B |= (1 << CS12) | (1 << CS10); // syntax for setting prescaler to 1024
  TIMSK1 |= (1 << OCIE1A); // enable compare match interrupt

  while(digitalRead(4) == HIGH); // waiting for switch to connect in order to begin data collection
  Serial.println("Data Collection has begun"); // message is sent to Serial Monitor to let the user know data collection has begun
  TCNT1 = 0; // resetting timer count for the timer interrupt
  sei(); // enabling interrupts
}

void loop()
{
  uint8_t number_of_data_points = 1 + 60; // first data point is discarded so then there's 60 left (1 for each second long interval of minute long trial)
  if(timer_cycles == number_of_data_points) // returns true once the trial is completed
  {
    cli(); // disabling interrupts since trial was completed
    Serial.println("Data Collection has ceased:");
    Serial.println("===== HALL SENSOR DATA =====");
    Serial.print("Counts:");
    for(uint8_t i = 0; i < number_of_data_points; i++) // looping through all 60 stored samples from the completed trial
    {
      Serial.print(" ");
      Serial.print(hall_sensor_array[i]); // displaying the hall sensor pulse count for each second long interval of trial
    }
    Serial.println("");
    Serial.print("RPM:");
```

```
    for(uint8_t i = 0; i < number_of_data_points; i++) // looping through all stored samples from the completed trial
    {
      Serial.print(" ");
      Serial.print(hall_sensor_array[i] * 15); // displaying the RPM for each second long interval of trial
    }
    Serial.println("");
    Serial.println("");
    Serial.println("===== GENERATOR DATA =====");
    Serial.print("ADC:");
    for(uint8_t i = 0; i < number_of_data_points; i++) // looping through all 60 stored samples from the completed trial
    {
      Serial.print(" ");
      Serial.print(generator_array[i]); // displaying the generator ADC readings for each second long interval of trial
    }
    Serial.println("");
    Serial.print("Voltage:");
    for(uint8_t i = 0; i < number_of_data_points; i++) // looping through all 60 stored samples from the completed trial
    {
      Serial.print(" ");
      Serial.print(generator_array[i] * 5.0 / 1023.0); // displaying the generator voltage readings for each second long interval of trial
    }
    sei(); // enabling interrupts
  }
}

void hall_sensor_interrupt() // hall_sensor_interrupt is the external interrupt function which will trigger each time the magnet passes by
the hall sensor
{
  hall_sensor_counts++; // incrementing hall_sensor_counts because the magnet passes by the hall sensor
}

ISR(TIMER1_COMPA_vect) // defining the Timer interrupt function
{
  hall_sensor_array[timer_cycles] = hall_sensor_counts; // storing the number of hall sensor pulses counted during the previous 1 second
interval
  generator_array[timer_cycles] = analogRead(A0); // reading the analog to digital conversion value of the generator's voltage output
  timer_cycles++; // incrementing because the timer interrupt was just triggered
  hall_sensor_counts = 0; // resetting the hall sensor trigger count so the next interval starts from zero
}
```